

Taylor & Francis
Taylor & Francis Group

Systems Science & Control Engineering

An Open Access Journal

ISSN: 2164-2583 (Online) Journal homepage: www.tandfonline.com/journals/tssc20

An interpretable dual-level lightweight framework for malaria diagnosis for enhanced remote diagnosis in resource-constrained settings

R. Deepika & T.S. Pradeep Kumar

To cite this article: R. Deepika & T.S. Pradeep Kumar (2025) An interpretable dual-level lightweight framework for malaria diagnosis for enhanced remote diagnosis in resource-constrained settings, Systems Science & Control Engineering, 13:1, 2579988, DOI: 10.1080/21642583.2025.2579988

To link to this article: https://doi.org/10.1080/21642583.2025.2579988

© 2025 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group.

Published online: 03 Nov 2025.

Submit your article to this journal

Article views: 975

View related articles

View Crossmark data

Citing articles: 2 View citing articles

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=tssc20

SYSTEMS SCIENCE & CONTROL ENGINEERING: AN OPEN ACCESS JOURNAL
2025, VOL. 13, NO. 1, 2579988
https://doi.org/10.1080/21642583.2025.2579988

Taylor & Francis
Taylor & Francis Group

OPEN ACCESS Check for updates

An interpretable dual-level lightweight framework for malaria diagnosis for enhanced remote diagnosis in resource-constrained settings

R. Deepika and T.S. Pradeep Kumar

School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, India

ABSTRACT

Certain mosquito species can transmit the deadly disease malaria to humans. During an outbreak, massive data storage and remote diagnosis are necessary. A possible solution is cloud computing, however even while it offers the possibility of remote monitoring and storage, it has a high latency. Whereas fog computing reduces latency to a greater extend and makes remote diagnosis easier. Our proposed framework with two lightweight models for diagnosis of malaria has addressed these issues. The level-1 diagnosis make use of Optimized Lightweight MLP. On the basis of the symptoms, the most likely cases are identified by the model and the result is forwarded to the healthcare provider who will serve as the intermediary. The proposed model in phase 1 has been combined with Anchor, an explainable AI (XAI), to aid in the diagnosis. The suggested model outperformed current models in terms of efficiency, achieving an impressive 96% accuracy rate. Strong model performance is indicated by the training and test curve's good AUC (Area Under the Curve) of 0.99. A lightweight customized CNN model with GradCAM assists in finding confirmed cases in the second level. Comparatively, our proposed lite-weight CNN model showed greater efficiency with fewer parameters, a smaller model size and equivalent or more accuracy of 95% achieving a good balance between computational efficiency and performance. Data augmentation and Image processing techniques improved the efficiency of the model. Finally, a simulation that mimics the operations of the proposed framework has been carried out using iFogSim.

ARTICLE HISTORY
Received 7 July 2025
Accepted 20 October 2025

KEYWORDS
Deep Learning (DL); malaria; fog computing; convolutional neural network (CNN); explainable AI (XAI); multi-layer perceptron (MLP)

1. Introduction

One of the most significant health issues in tropical and subtropical regions is malaria. In current world, the diagnosis of Malaria is carried out in two steps. The first step deals with finding the probable cases infected with malaria based on the symptoms. Clinical manifestations of simple malaria include exhaustion, nausea, fever, vomiting, headaches and muscle aches. The feverish disease, malaria usually develops 10 to 15 days following a mosquito bite. Due to its unpredictable and ambiguous clinical symptoms, doctors who rarely treat malaria may misdiagnose it as encephalitis, viral hepatitis, dengue, influenza, or gastroenteritis. Differentiating malaria from other feverish illnesses requires specific diagnostic methods like blood smear test. Early malaria discovery can stop the progression and lessen the severity of the disease (Harmonis et al., 2025). During an epidemic, the number of infected cases is huge. Hence, there is a need to automate it to find the probable cases. The second step deals with confirmation of those probable cases based on the analysis of blood smear. Rapid diagnostic tests (RDTs), PCR-based techniques, or blood smear microscopy are used to diagnose malaria. Because of their complexity and cost, PCR-based methods have not been extensively used in areas where malaria is common. Since cases can be identified in 15 to 30 min and sophisticated PCR equipment or skilled specialists are not needed, malaria RDTs are helpful diagnostic tools. Malaria RDTs identify the presence of antigens, which are particular proteins produced by malaria parasites in blood of the infected people. Low specificity, an inability to quantify parasite density, susceptibility to heat and moisture, and higher costs when compared to microscopy-based techniques are some of the drawbacks in RDTs based on reports from World Health Organization (WHO) (Harmonis et al., 2025). Despite being widely used, traditional diagnostic techniques like microscopic analysis of blood smears are laborious and prone to human error. To get around these problems, a lot of research has

CONTACT T.S. Pradeep Kumar tspradeepkumar@vit.ac.in

© 2025 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group.
This is an Open Access article distributed under the terms of the Creative Commons Attribution License (http://creativecommons.org/licenses/by/4.0/), which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited. The terms on which this article has been published allow the posting of the Accepted Manuscript in a repository by the author(s) or with their consent.

been done on the use of computer-aided diagnosis systems for automatic malaria detection and classification (P. Kaushik & Sharma, 2025, January). There are several types of malaria diagnostic methods available at the moment. To detect Plasmodium parasites, dyed thick and thin blood films are examined under a microscope. Because of their affordability, ease of preparation, and reduced time requirements, microscope evaluations are becoming the gold standard.

1.1. Research gap

Every year, millions of blood films are analyzed for malaria which means a skilled microscopist must meticulously count parasites and infected red blood cells. However, because microscope examinations rely so significantly on the knowledge and abilities of parasitologists, the results are rarely standardized (Sukumarran et al., 2024). As false positives might result in patients developing drug resistance following needless treatment and false negatives can be lethal, the accuracy of a diagnosis is very important. The last requirement is laboratory confirmation of blood smear microscopies for diagnosing the malaria infection. Traditionally, medical professionals performed the examination, which is difficult and prone to mistakes (Chen et al., 2023). A patient with malaria needs proper medical attention. Thus, in order to enable accessible healthcare services through the use of emerging technologies like fog computing, artificial intelligence (AI) and cloud computing, a remote diagnostic system is necessary (Vijayakumar et al., 2019).

1.2. Problem statement

Fog computing (FC) is a decentralized approach to data processing and storage close to the source, which enhances network device performance generally and speeds up reaction times. By adapting fog computing, latency can be reduced and performance can be optimized (Ayalew et al., 2024). Latency is a major concern with cloud computing, though it offers a way to get over storage problems. Hence the fog layer has been introduced between the edge and the cloud layer to overcome the issue. Instead of sending all the data to the cloud for processing, data is sent to fog which is closer to the edge layer. For real-time monitoring and prompt response to possible malaria cases, this reduces latency and makes early treatments possible (Ayalew et al., 2024). Because of these significant benefits, FC is used as the middle layer in the proposed system. The fog layer is used for rapid diagnosis and emergency alarms. The use of artificial intelligence (AI) techniques can lead to precise disease detection, and the interconnected architectures provide accurate reporting of patient status. Additionally, the majority of studies focused on the first stage of diagnosis, which aids in identifying the most likely instances however blood smear test results are used to confirm the disease. In the fog computing environment, a lightweight model is also required. Because of the vast volume of real-time data produced by devices, machine learning algorithms are essential for remote healthcare. When properly trained, they can extract relevant information with high accuracy.

1.3. Research contributions

By identifying confirmed cases this work aims to provide a framework for diagnosing malaria at two levels using explainable AI, that facilitates distant diagnosis and reduces the number of inappropriate medical interventions. Our proposed framework has addressed these issues. Some of the findings include,

- Optimized and normalized lightweight MLP model along with lightweight Explainable AI (XAI) for the diagnosis of probable malaria cases in the fog computing environment based on symptoms
- Lightweight EfficientNet model along with lightweight XAI for the diagnosis of confirmed cases
- Framework for Dual level diagnosis of malaria that mimic the real-world scenario.

2. Related works

The limits of conventional approaches in malaria diagnosis and the growing significance of deep learning in disease diagnosis are reviewed in this section. The related work section is divided into four sections: Conventional approaches for malaria diagnosis and drawbacks, Machine Learning (ML) and Deep Learning

(DL) approaches for malaria diagnosis, lightweight ML/DL approaches and explainable AI (XAI) for disease diagnosis, fog computing framework for healthcare.

2.1. Conventional approaches for malaria diagnosis and the drawbacks

Rapid diagnostic testing (RDT) or microscopic examination of blood smears are the primary methods used to diagnose malaria. Malaria diagnosis relies on microscopy to detect and identify Plasmodium parasites in blood smears. Despite being widely used, the efficacy of the technique is limited by inter-observer variability, the labour-intensive nature of the study, and the requirement for specialized workers (Benachour et al., 2025). The quality of the microscope, fixation, and blood smear staining all impact the time-consuming outcomes of microscopic diagnosis. Additionally, RDT is employed when microscopy is impractical. Additionally, prolonged stability of histidine rich protein 2 (HRP2) in blood may result in false-positive results, and RDTs are limited in cases of low-density parasitemia. However, because health records are not properly stored, tribal members and villagers in high-malaria endemic areas are often exposed to malaria infections but are unable to reproduce their medical history, including the duration of infection, the type of malaria parasite, the recommended treatment, and the outcome (Nema et al., 2021). As treating false positive patients develops antimalarial medication resistance and untreated false negative patients are the reason for the spread of malaria, these medical data may be crucial to both treating and halting the increase in drug resistance. Furthermore, the lack of healthcare facilities close to their villages, loss of income, and cultural beliefs all contribute to the tribal people's reluctance to seek medical attention, which makes it more difficult to control and manage malaria in remote areas. Significant stock outs, delays in issuing new stock, and inconsistent reporting of consumption of critical malaria commodities all impact service preparedness and result in patient downtime (Nema et al., 2021).

2.2. ML and DL approaches for malaria diagnosis

Research on diagnosing malaria has concentrated on enhancing the speed and precision of the diagnostic tools while also making them more accessible in malaria-affected areas. With the introduction of automated microscopy tools, a more uniform method for identifying malaria parasites in blood samples has been made possible. These technologies, which make use of ML algorithms and sophisticated imaging techniques, have demonstrated the potential to lower the human error involved in conventional microscopy. When combined with microscopy, artificial intelligence and machine learning-based algorithms can analyze blood smears and enable rapid morphological-based malaria parasite detection. Even with these advancements, there are still a lot of challenges to solve. Hemozoin-based digital malaria diagnostic tools can be used to enhance diagnostic processes. Therefore, in rural and tribal areas, the use of easily transportable and accessible internet platforms for malaria could be advantageous for the medical professional who works there to actively treat the condition. Additionally, community health providers can use the idea of telemedicine in emergency situations to give the patient quick relief and then direct them to an advanced care centre (Nema et al., 2021). There are two methods for diagnosing malaria. The first one is diagnosis of the disease based on the symptoms (Bria et al., 2021), and the other method is the identification of the parasite in the blood smear of the probable cases (P. Kaushik & Sharma, 2025, January). ML/DL approaches help in the automation of the above methods (Fuhad et al., 2020). Only the most probable instances of malaria can be identified by clinical diagnosis. Identification of the parasite in the blood smear sample of the infected person is necessary for case confirmation. Novel CNN models have been proposed in both papers (Gourisaria et al., 2020) and (Siłka et al., 2023) to diagnose malaria from the blood smear images. Table 1 below lists a few studies that focus on diagnosing malaria using ML and DL approaches.

2.3. Lightweight ML/DL approaches with XAI

Deep learning models that are lightweight have become crucial in malaria classification because they can function well on gadgets with low processing power, including smartphones and embedded systems. Using deep learning, by integrating CNN for classification and IGMS for screening, Yang et al. (2019) developed a smartphone app that can detect malaria parasites. They are the first to utilize deep learning on cellphones for

Table 1. Table of various ML/DL approaches in malaria diagnosis.

Ref	Dataset	Proposed model	Symptom based diagnosis	Microscopic images-based diagnosis	Parameters (includes trainable)	Future work
Kaushik and Sharma (2025, January)	27,558 microscopic cell images from Kaggle	Convolution Neural Network (CNN)	No	Yes	622,817	To integrate Explainable Artificial Intelligence (XAI) and improve compatibility with the edge devices
Sora-Cardenas et al. (2025)	One thousand thick blood smear pictures dyed with Romanowsky from the Colombian National Institute of Health	Customized Convolution Neural Network (CNN)	No	Yes	$\sim$ 291,138	To Enhance accessibility in remote locations
Dwivedi et al. (2025)	Dataset is available at https://air.ug/datasets/ which includes 1182 colour pictures of thick blood smears stained with field stain (750 $\times$ 750 pixels).	YOLO-mp-3I model for intelligent suite with OpenVINO	No	Yes	11.19 million	To include XAI
Yang et al. (2019)	1819 thick smear images from 150 patients, Chittagong Medical College Hospital, Bangladesh	Iterative Global Minimum Screening (IGMS)based screening with CNN	No	Yes	313,906	To use Ensemble techniques and make even more compatible model for mobile phones
Bria et al. (2021)	Real time data collection from hospitals	Logistic Regression	Yes	No	Not Applicable	To develop a sophisticated malaria diagnostic method for early malaria detection
Mariki et al. (2022)	Real time data collection from hospitals	Random Forest	Yes	No	Not Applicable	To expand the dataset and add new regions to improve the model

thicksmears, and their technique is evaluated at the patient level Magotra and Rohil (2022) suggested a customized CNN architecture for malaria classification. Six max-pooling layers and the 'Relu' activation function come after the six layers of convolution, which have 16–128 is the range of filter sizes. To balance classification performance and computational economy, the model has 381,729 parameters, of which 381,313 (0.364 MB 8 bitwise) are trainable and 416 are not. A lightweight deep learning model for identifying malaria parasite types and classifying life cycle stages was presented by Chaudhry et al. (2024). With fewer than 0.4 million parameters, it is more than 20 times lighter than DenseNet. Their model was developed especially for use in mobile apps, especially in low-resource environments, and it performed better than previous approaches. Four distinct malaria datasets were used for testing. A lightweight CNN model was proposed by Al Muazzaz and Goni (2024, September) for the early-stage detection of malaria parasites, and it achieved 99.56% accuracy, 99.97% precision, 99% F1-score, 99.2% AUC score, and 98.36% recall for early detection of malaria from microscopic blood pictures.

A prediction time of two milliseconds per image was used to illustrate the efficacy of the model. To improve interpretability, infected regions were highlighted using Grad-CAM. Because of its portability, the model can be effectively implemented in environments with limited resources, improving healthcare efficiency and accessibility. In order to diagnose malaria from RBC pictures, A modified lightweight CNN with 0.17M parameters was proposed by Goni and Mondal (2023, June) and yielded 99.45% accuracy, 99.75% precision, 99.17% recall, and 99.46% F1-score. The model surpassed SOTA and transfer learning models in terms of performance and efficiency when assessed using SHAP for explainability, which makes it appropriate for quick and accurate malaria detection. S. Khan et al. (2025) addressed the interpretability and computational inefficiency of current models by suggesting a fractional-order optimizer-based lightweight CNN for malaria diagnosis. The model demonstrated robustness with 95% accuracy on the NIH dataset and 92% and 90.4% accuracy on the MP-IDB and M5 test sets, respectively (https). Using an open-source dataset of 26,161 blood smear images, Salam et al. (2024) automatically diagnosed malaria using a lightweight 17-layer SqueezeNet model with 119,154 trainable parameters. The model demonstrated remarkable performance metrics on the original NIH dataset, including 96.37% accuracy, 95.67% precision, 97.21% recall, and 96.44% F1 score. It was tuned for embedded systems. It was just 1.72 MB in size. Without model interpretability, attaining high accuracy in health risk forecasting is not enough for real-world application. Techniques like Explainable Artificial Intelligence (XAI) have become essential tools to address issue. By using these XAI techniques, we are able to pinpoint the main variables that influence the model's assessment, improving model transparency and fostering confidence in the forecasted results. SHapley Additive Explanation (SHAP), Gradient-weighted Class Activation Mapping (Grad-CAM), and Local Interpretable Model Agnostic Explanation (LIME) are XAI techniques used to better analyze the data. Grad-CAM is used to generate heatmaps, SHAP to shape the values, and LIME to forecast the outcomes (Koul et al., 2024). The Local Interpretable Model Agnostic Explanations (LIME) framework is used in this study to clarify how the suggested model is decided. LIME provides insights into the elements driving a given outcome by fitting a simplified, interpretable model that approximates the complicated model behaviour locally around a certain classification (Azad et al., 2025). A few other related works that employ lightweight models with or without XAI have been discussed below in Table 2.

2.4. Fog computing framework for disease diagnosis

A small delay or unavailability of an application for whatever reason (e.g. power outage, Internet outage, cloud failure, etc.) may be lethal in some circumstances (A. Kumari et al., 2018). It is unacceptable to slow down data transfer, process it in the cloud, and then instantly deliver the results to users. In certain situations, a minor delay or unavailability of an application for whatever reason (such as an interruption in power, a loss of Internet access, a cloud failure, etc.) could be fatal (A. Kumari et al., 2018). Fog computing (FC), a recently developed computing paradigm, is established between IoT devices/edge layer and CC in order to address the aforementioned problems of Cloud Computing (CC). The intelligent gateway processes the sensed data and sends it to either CC or FC. Fog gateway allows professionals to manage emergency situations with the least amount of delay in information collection. A robust communication infrastructure is always required for improved performance against delay. In interactive environments, FC is used to make it easier for devices to get the necessary data through a cloud network. The CC paradigm is extended to the edge of the network by FC. There are numerous studies of applying fog computing models in practical applications in addition to

Table 2. Table of various lightweight ML/DL approaches and XAI for Disease diagnosis.

Reference	Lightweight model and techniques	Disease	XAI used	XAI techniques	Trainable parameters	Future work
Mariki et al. (2022)	Lightweight EfficientNet-B3	Leukemia Detection	No	–	11, 093, 290	To include XAI
Hasan et al. (2024)	Lightweight Multi-Scale CNN	Lung and Colon cancer detection	Yes	GRADCAM, SHAP	1, 100, 000	To minimize errors in XAI
Assaduzzaman et al. (2025)	EfficientNetB0 with Squeeze Excitation (SE) blocks and multi-scale feature fusion	Tomato Leaf disease	Yes	SHAP, LIME, GRADCAM, GRADCAM++	5, 330, 564	To develop more affordable solutions for problems related to agriculture
S. Khan et al. (2025)	Modified EfficientNet-B0	Pulmonary disease	Yes	GRADCAM, t-SNE	16, 378, 536	To include human-in-a-loop validation process with additional imaging techniques like CT scans
Iqbal et al. (2024)	Lightweight deep network model	Coronary artery disease	Yes	GRADCAM and Layer-Wise Relevance Propagation (LRP)	507, 299	To explore 3D-CNN model in future
Bhandari, Shahi, et al. (2023)	Lightweight CNN Model	Renal abnormalities: cysts, stones and Tumours	Yes	LI LIME, SHAP	182, 916	To improve the performance using data augmentation
Bhandari, Yogarajah, et al. (2023)	Lightweight custom CNN Model	Retinal disease	Yes	LIME, SHAP	983, 000	To test the model with more real time dataset
	Proposed CNN Model	Malaria	Yes	Anchor (LIME), GRADCAM	127, 682	To test the model with different dataset

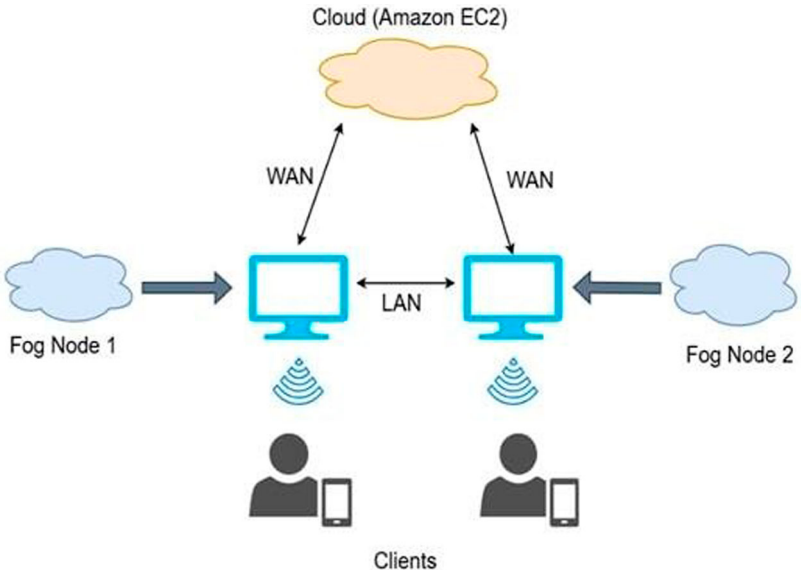


Figure 1. A simple fog computing framework for predicting diseases.

Table 3. Table showcasing the application of various technologies in different lightweight ML/DL approaches for Disease diagnosis.

Reference	Disease	Model used	CC	FC	EC	SM	AG	RTA
Minh et al. (2023)	Heart	CNN-LSTM	Y	Y	N	N	Y	Y
S. Kumari et al. (2022)	Encephalitis	M-LSTM	Y	Y	N	N	Y	Y
Verma et al. (2022)	Heart	DNN ensemble	Y	Y	N	N	N	Y
Yu and Reiff-Marganiec (2021)	Skin disease	Targeted Ensemble Machine Classify Model (TEMCM)	Y	Y	N	Y	Y	N
Q. Zhang et al. (2019)	General	Multi-modal deep computation model	Y	N	Y	Y	N	N
Anuradha et al. (2021)	Cancer	DNNs	Y	N	N	Y	N	N
Juyal et al. (2021)	Skin disease	Hierarchical CNN	Y	N	N	Y	Y	N

the theoretical study in the field where prototypes based on fog computing have been developed to enable cloud-deployed applications. Applications based on fog computing have been extensively researched recently for expert systems for the diagnosis of heart disease. Neural networks can identify and categorize ECG signal analysis in various applications (Minh et al., 2023). Below, Table 3 describes other related works that incorporate fog computing frameworks for disease prediction. A basic fog computing framework has been depicted in Figure 1. Input data are received from the edge devices by the fog nodes and further processing of the data is done in the fog node. The model is placed in the fog node instead of placing it in the cloud. This reduces the latency and enables speedy processing of data. A basic fog computing framework has been depicted in Figure 1. From the edge devices by the fog nodes, the input data are recieved and further processing of the data is done in the fog node. The model is placed in the fog node instead of placing it in the cloud. This reduces the latency and enables speedy processing of data. The fog node transmits the results back to the edge devices and stores them in the cloud for further use.

3. Materials and methods

This section includes information about the datasets and the methods.

3.1. Datasets

In our proposed framework, we have used two different models in fog computing environment for dual level diagnosis of malaria. A model is trained using a collection of cell images, while symptoms are used to train the other. We have proposed a lightweight improved Multilayer perceptron (MLP) model for symptom-based

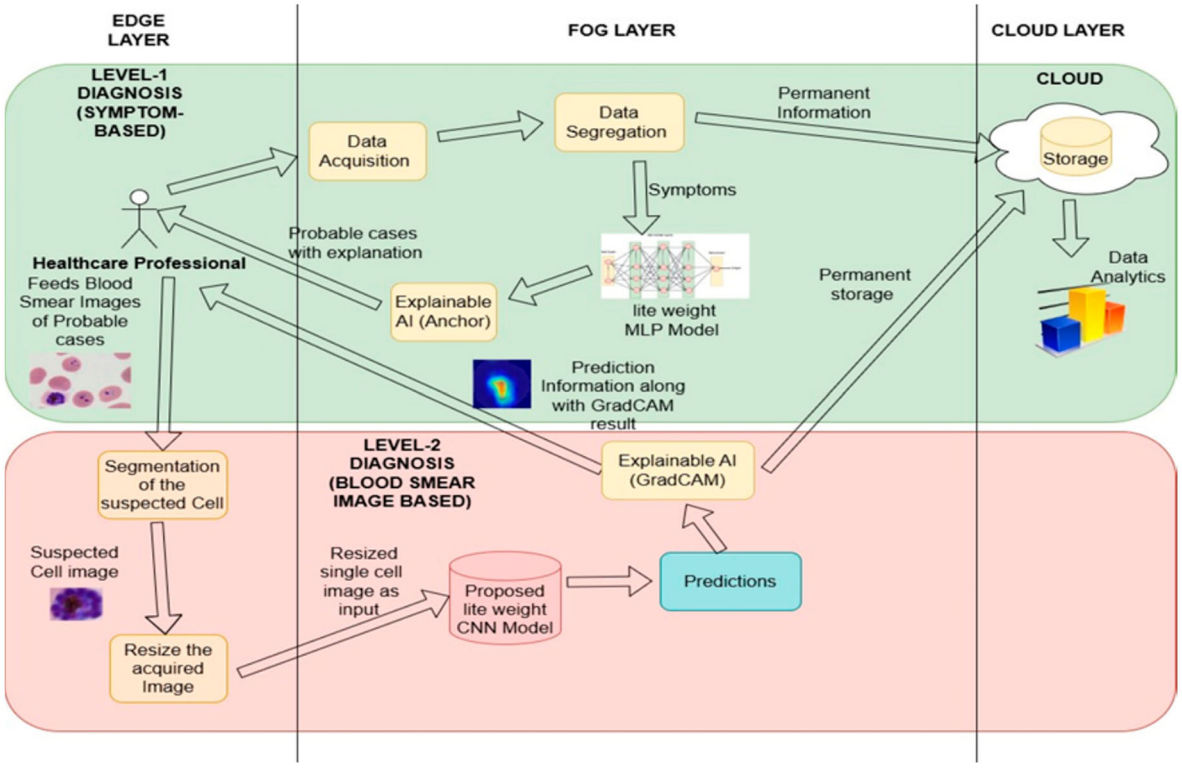


Figure 2. Proposed framework for dual-level diagnosis of malaria.

diagnosis of probable cases infected with malaria. Also, we have proposed a lightweight EfficientNetB0 model for second level diagnosis of malaria based on blood smear images. These two models are trained using two different datasets. For the symptomatic diagnosis of malaria, we have used the secondary dataset provided by the 'The Federal Polytechnic Ilaro Medical Centre' in Ilaro, Ogun State, Nigeria which includes information on 337 patients who sought consultation for symptoms linked to malaria. After the patients were tested for malaria, information about them was gathered, and the symptoms they reported were documented. These patients range in age from 3 to 77 years, with 180 of them being female and 157 being male. Data was gathered for four weeks. This dataset includes 15 symptoms of malaria. The patients' genders are encoded in ordinal form as '0' for males and '1' for females, and their ages are given in years from the dataset. Numbers are used to represent other attributes, with '0' signifying the lack of symptoms and '1' signifying their presence. The medical director, who is the representative of the institutional bioethics committee has authorized this raw dataset, which is readily available and regarded as supplemental data (Adeboye et al., 2020). The dataset for second level of diagnosis has been taken from Kaggle which includes 27,558 microscopic cell images split into two categories: 13,780 photos were divided into two classes: Parasitized (Infected) and Uninfected. These high-resolution blood smear photos have been captioned according to the presence or absence of the malaria-causing Plasmodium parasite. Due to the dataset's balance and equal representation of both groups, bias in model evaluation and training is eliminated. Brightness, staining, and image quality variations are examples of varied image variations that enable the model to generalize to previously encountered data with effectiveness. For training ML models, this large and well-balanced dataset is ideal (https)

3.2. Methods

This section explains the various methods and techniques carried out in two levels for efficient diagnosis of Malaria. The proposed dual level framework has been proposed in the Figure 2.

3.2.1. Level-1 diagnosis for identifying the probable cases based on symptoms

In the level-1 diagnosis, the information about the patient and symptoms are transmitted to the fog layer collectively. After the data acquisition module in the fog layer gets the data, it is sent to the data segregation module. It keeps patient data separate from data pertaining to the illness. The proposed lightweight Multi-Layer Perceptron (MLP) model receives the data pertaining to the symptoms, and the cloud receives the permanent data gathered for storage. Lightweight explainable AI (XAI) is then used to explain the predictions. From the fog layer, the medical professional is informed about the probable cases and given an explanation.

3.2.1.1. Preprocessing and data analysis. One crucial phase in the machine learning pipeline is preprocessing. It is crucial for preserving the consistency of the data. Missing values, inaccurate entries, and other irregularities are examined throughout the entire dataset (Deepika & Pradeep Kumar, 2025). In the Federal Polytechnic Ilaro Medical Centre's dataset, two columns 'age' and 'sex' which was not contributing any outcome for the target variable has been removed. The entire dataset has been checked for null entries. After preprocessing steps, analysis of the data has been carried out. Relationship between the independent variable and the target variable has been analysed using Pearson correlation co-efficient technique. A heat map was developed using the Pearson correlation co-efficient technique. Heat maps are a powerful exploratory data analysis (EDA) tool that facilitates pre-processing by quickly seeing the relationships and patterns of the dataset (Singh et al., 2024). The Pearson correlation coefficient (r) can be found using the formula in Eq. (1) below.

$$r = \sum \left(x_i - \bar{x} \right) \left(y_i - \bar{y} \right) \sqrt{\left(\sum \left(x_i - \bar{x} \right)^2 \sum \left(y_i - \bar{y} \right)^2 \right)} \tag{1}$$

The variables X and Y are represented by the data points x_i and y_i, where the mean of the X values is $\bar{x}$ and the mean of the Y values is $\bar{y}$.

$\sum$- Total sum of all data points.

r has a range of -1 to $+1$.

A complete positive linear connection, or one in which Y increases proportionately to X, is denoted by $r = +1$.

$r = -1$ denotes a perfect negative linear relationship meaning that Y reduces proportionately as X grows.

There is no linear relationship when $r = 0$.

The target variable in the dataset is 'severe_maleria' and the rest of the variables are independent variables. So, in Figure 3, symptoms like jaundice, fever, rigour, fatigue, cocacola_urine, cold, diarrhoea, hypoglycemia, headache are positively correlated to the target variable whereas, other variables like bitter_tongue, vomiting etc. are negatively correlated. In Table 1, the correlation coefficient values between the target feature and the independent features are given. Figure 4 depicts the correlation values between each independent variable with the target variable. The independent variables whose values are above 0, are positively correlated to the target variable. Whereas the independent variables whose values are below 0, are negatively correlated to the target variable.

3.2.1.2. Various ML and DL models. Various ML and DL models were trained and tested with the dataset from the Federal Polytechnic Ilaro Medical Centre. Among all, the proposed lightweight MLP gave promising results. As high accuracy is a crucial requirement in any diagnostics, our proposed MLP model gave a satisfying accuracy of 96% for a relatively small dataset. Above, Table 2 displays the list of recent lightweight ML/DL approaches and XAI for Disease diagnosis.

3.2.1.3. Proposed lightweight MLP model. Multilayer perceptrons (MLPs) are artificial neural networks that have many node levels, such as input, output, and one or more thick layers. It is a powerful model that can identify complex patterns in both organized and unstructured data. The proposed lightweight MLP model is composed of four dense layers. The output layer is one of the four, while the other three are hidden. While the three hidden layers are made up of ReLU activation functions, a sigmoid activation function is used to perform binary classification in the output layer.

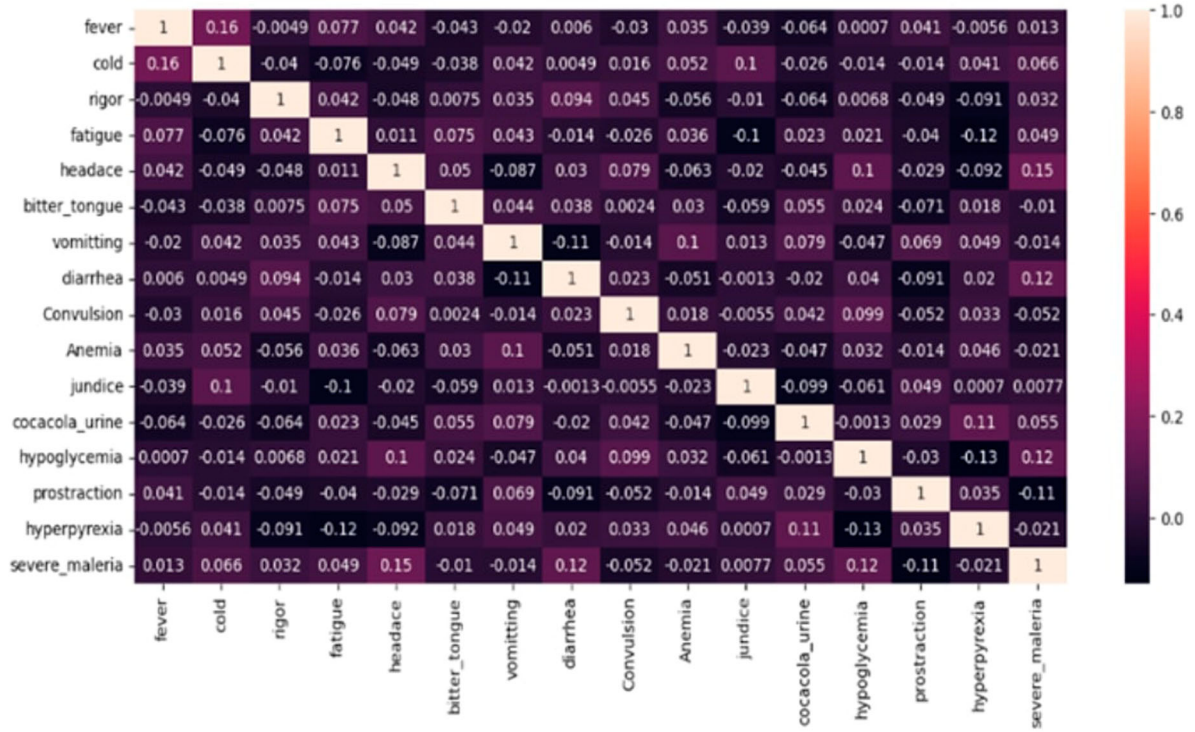


Figure 3. Heatmap based on correlation co-efficient values.

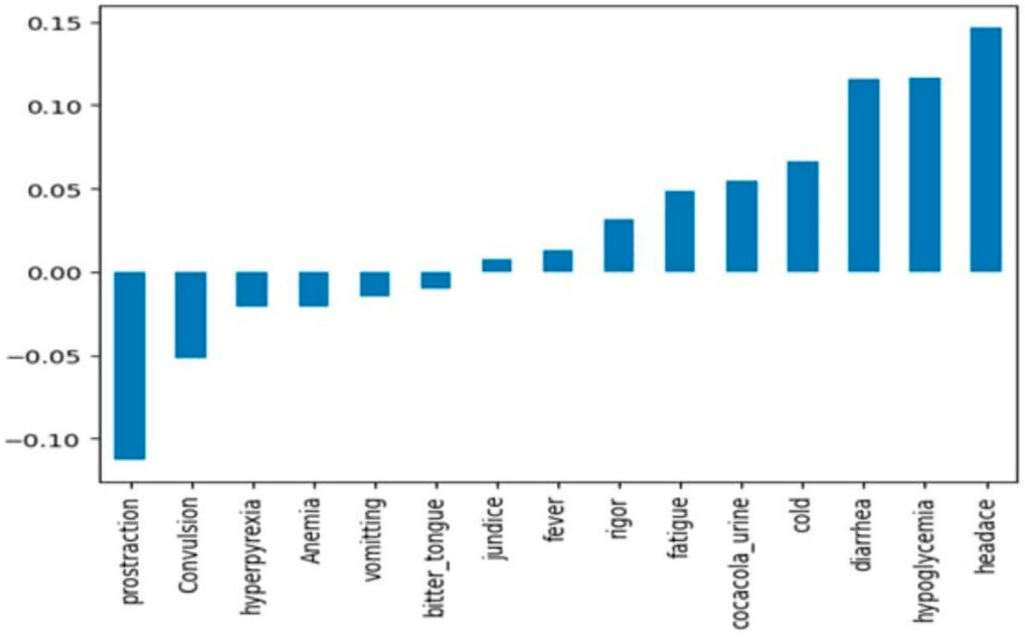


Figure 4. Bar graph showing values of the correlation between the target variable and the independent variables.

3.2.1.3.1. ReLU and sigmoid activation function. The mathematical definition of ReLU (Rectified Linear activation function) is provided by Equation (2) below.

$$f(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases} \tag{2}$$

where, if x is positive, the output is x. Additionally, if x is 0 or negative, the outcome is 0. It introduces non-linearity by 'clipping' all negative inputs to zero although the ReLU function seems linear for positive values.

This non-linearity allows neural networks to learn complex patterns. The sigmoid activation function is widely used in neural networks, particularly in the output layer for binary categorization tasks. It is also known as the logistic function. The formula in Equation (3) defines the sigmoid function.

$$f(x) = \frac{1}{1 + e^{-x}} \tag{3}$$

where x serves as the input to the sigmoid function and its output falls in the range 0 to 1. When normalized probabilities are required, sigmoid performs perfectly. It creates an S-shaped curve. It is hence referred to as the sigmoid function. For positive inputs, the curve asymptotes to 1, for negative inputs the curve asymptotes to 0, and for values close to 0, the curve asymptotes to 0.5. During training, dropout randomly sets a subset of input units to zero in order to prevent overfitting (Deepika & Pradeep Kumar, 2025).

3.2.1.3.2. Batch normalization. By levelling activations, batch normalization accelerates convergence and enhances generalization. The rate of convergence of the model is accelerated and issues like internal covariate shift are mitigated by normalizing the inputs to a layer for every mini-batch during training. It normalizes the inputs during training, assigning a mean of 0 and a standard deviation of 1 to each mini-batch (Deepika & Pradeep Kumar, 2025).

3.2.1.3.3. Drop out. Dropout is a regularization technique used by neural networks to prevent overfitting. A randomized 'dropping out' (i.e. setting to zero) specific neurons occurs with each training session. The model needs to build stronger and broad characteristics because it can no longer rely on specific neurons being present constantly. The Drop Out functions in two ways. Dropout deactivates neurons at random during training. Eq. (4) provides a mathematical demonstration of this.

$$\text{Output}_i = \begin{cases} 0, & \text{with probability } p \\ \text{Activation}_i, & \text{with probability } 1 - p \end{cases} \tag{4}$$

The dropouts are not used during testing. Rather, to maintain the same anticipated result, the activations are scaled by (1−p) (Deepika & Pradeep Kumar, 2025).

3.2.1.3.4. Adam optimizer. Adam optimizer is combination of two other popular optimization techniques: Momentum (using prior gradients to smooth updates) and AdaGrad (adaptive learning rates). Adam, an acronym for Adaptive Moment Estimation, determines adaptive learning rates for each parameter by estimating the gradients' first moment (mean) and second moment (variance). The Adam optimizer is a versatile, efficient, and flexible optimization method (Deepika & Pradeep Kumar, 2025).

3.2.1.3.5. Binary cross-entropy. Binary cross-entropy measures the variation between the anticipated probability distribution and the true label distribution for binary classification problems. It evaluates the degree to which the actual labels correspond to the model's predicted likelihood. Here, our proposed model makes use of the loss function. Its objective is to determine the extent to which the expected likelihood and the actual binary labels (0 or 1) match (Deepika & Pradeep Kumar, 2025).

3.2.1.3.6. GridSearchCV for hyperparameter tuning. GridSearchCV is used to systematically find the best a group of hyperparameters for a given model. It thoroughly searches a predefined grid of hyperparameters before evaluating the model for every combination using cross-validation. GridSearchCV is a reliable and systematic method for optimizing model hyperparameters. GridSearchCV primarily carries out three processes: hyperparameter tweaking, cross validation, and rigourous search, which examines every potential grid hyperparameter combination. It can be computationally challenging for large search spaces, but it ensures that the optimal parameters are found. For larger or more complex problems, alternatives like Bayesian optimization or RandomizedSearchCV can be more suitable (Deepika & Pradeep Kumar, 2025). We have compared the effects of different optimization techniques during modelling (Level-1 diagnosis) The comparison is shown below in Table 4. Based on the comparison, our proposed optimization technique (Grid search with cross validation) proves to be efficient. As grid search is exhaustive, deterministic, reproducible, explainable, and appropriate for limited datasets/parameter spaces, it is frequently used in medical artificial intelligence. Grid Search is more suited to the highly regulated nature of medical applications, even though Optuna/Bayesian are efficient.

3.2.1.3.7. K-fold cross validation. It divides the dataset into K number of subsets, or folds, in order to evaluate the model's performance. By evaluating the ability of the model to generalize to different dataset, it reduces the likelihood of overfitting. It splits the dataset into K folds for each of the K iterations, uses one-fold for testing, and then trains on the remaining k-1 folds. After K iterations, average the performance measure across all folds

Table 4. Comparison between effects of different optimization techniques during modelling (Level-1 diagnosis).

Optimization techniques	Accuracy	Precision	Recall	F1 Score
Bayesian optimization	0.83	0.88	0.83	0.83
Randomized search	0.92	0.98	0.83	0.91
Optuna	0.67	0.80	0.44	0.57
Proposed Gridsearchcv	0.96	0.95	0.90	0.92

to get an overall estimate. When modelling in this instance, five-fold cross-validation was carried out (Deepika & Pradeep Kumar, 2025).

3.2.1.3.8. Dense layers and output layer in the proposed lightweight MLP model. There are three intermediates completely linked dense layers and an output dense layer fully integrated with the Sigmoid activation function for binary classification tasks.

- Dense Layer 1: The first dense layer's input_shape parameter defines the input layer. This thick layer contains a large number of 'unit1' neurons. When we use GridSearchCV, we replace the variable 'unit1' with the values found in the hyperparameter grid. ReLU activation is applied to all feature input.
- Dense Layer 2: In GridSearch, 'unit2' neurons are substituted for the variable 'unit2' in this dense layer with the values found in the hyperparameter grid. Input is provided by neurons in the layer above. ReLU serves as the activation function for this layer.
- Dense Layer 3: In GridSearch, 'unit3' neurons are substituted for the variable 'unit3' in this dense layer with the values found in the hyperparameter grid. ReLU serves as the activation function for this layer.
- Output Layer: This fully connected layer consists of a single neuron with a sigmoid activation curve for binary classification.

A dropout function with a dropout rate of 0.3 comes after Dense Layer 1. Additionally, batch normalization is done following the dropout. However, only the drop-out function is executed following dense layers two and three. The loss function is binary cross-entropy, and the output layer's Adam optimizer is used for optimization after the output operation. The degree to which the actual labels correspond to the model's expected probability is measured by binary cross-entropy.

With batch size = 16, dropout_rate = 0.2, unit1 = 64, unit2 = 32, and unit3 = 16, the model performed well and yielded a 96% accuracy after GridSearchCV's hyperparameter tuning. Number of neurons in the dense layers are specified as unit1, unit2 and unit3. When compared to networks that are deeper or wider, the overall number of trainable parameters is rather small. As a result, the model is lightweight. The K-fold has been employed in the procedure, with n_splits set to 5. After counting the model's parameters layer by layer, 3,713 total parameters were employed. This model has 128 non-trainable parameters and 3,585 trainable parametersThe size of the model is roughly 14.50 KB. 200 epochs were carried out. Each epoch of training takes 0.8 seconds when using a standard CPU (central processing unit). Three minutes of CPU time were spent on training.

3.2.1.4. Enhanced explainability using ANCHOR in level-1 diagnosis.

3.2.1.4.1. Explainable AI (XAI). The objective of XAI techniques is to enable the decision-making processes of AI systems more transparent and understandable so they can express their advantages and disadvantages and offer insights into how they are likely to behave in different future situations. The XAI approach investigates and creates a variety of techniques, giving AI developers a wide range of design options. SHAP, LIME and Anchor are some of the XAI techniques used for local explanations. Table 5 below shows the comparison between these techniques based on different aspects. Based on the comparison, Anchor has been selected as the XAI technique for the proposed model as decision-making and high interpretability are the most important requirement in healthcare decision-making. With the help of these choices, developers can strike the ideal balance between an AI system's performance and its capacity to clearly describe its internal operations. XAI offers a variety of methods to satisfy a wide range of demands and preferences while maximizing the performance-explainability trade-off in accordance with the particular needs of any AI application (Sawangarreerak et al., 2025). In the work (Gipiškis et al., 2024), XAI in image segmentation has been explained. Approaches that are

Table 5. Comparison between different XAI techniques.

	LIME	ANCHOR	SHAP
Time consumption	Moderate	Slower than LIME	Slower than LIME
Usage	Any model	Models other than tree- based models	Small models
Interpretability level	Medium	Very high	High
Output	List of feature weights	If-then rules	Provides local and global explanation
BestChoice (in healthcare)	For exploratory prototyping	For patients and decision makers	For clinicians and regulators
AverageTime Consumption	Moderate	Moderate-High	High
Number of model calls (per explanation)	500–5000	1000-5000+	10000+

based on prototypes, gradients, perturbations, counterfactual scenarios, and architecture has been proposed in paper (Raisa et al., 2025).

3.2.1.4.2. Anchor. *By determining decision boundaries that adequately 'anchor' the prediction,* the Anchor techniques provide an explanation for the predictions of any black-box classification model. A rule that effectively connects a prediction to a specific local context inside the instance being studied is known as an anchor explanation. This rule ensures that the rule's capacity to explain the prediction is not significantly impacted by changes to other feature values. Through the use of graph search algorithms and reinforcement learning techniques, the Anchors strategy effectively recovers from local optima and minimizes the number of model calls required during runtime (Sawangarreerak et al., 2025). Anchor is an algorithm that uses a perturbation-based approach to generate local explanations for fundamental IF–THEN principles. Anchor's coverage strategy enables these rules to precisely explain any potential unseen cases. The Anchor method uses reinforcement learning techniques to solve the exploration or multiarmed bandit problem, which is used to find anchors. By examining the neighbours or perturbations of each instance under explanation, Anchor enables the algorithm to function without considering the internal structure and parameters of the black-box model. As a result, the properties and structure of the black box remain hidden and unchanged throughout the process (Sawangarreerak et al., 2025). The formal definition of an anchor is given below in equation (5).

$$E_{Dx((z|A))}[1_{\hat{f}(x)=\hat{f}(z)}] \geq \tau, A(x) = 1 \tag{5}$$

where, similar to a record in a dataset, x stands for the data point under examination. If the features given by A match the qualities of x, then A (x) = 1 holds true. A is a set of criteria or the created rule/anchor. The model being described, such as an artificial neural network, that forecasts result for x and its perturbations is indicated by the letter f. The distribution of cases near x that satisfy A is denoted by Dx (·|A). A precision threshold is established by the parameter $0 \leq \tau < 1$, and rules must attain a local fidelity at or above τ in order to be regarded as trustworthy explanations (Sawangarreerak et al., 2025). Figure 5 shown below explains how explanation for the model's prediction is given using Anchor. The model forecasts class 0 which depending on the labels, probably means no sickness or a negative diagnosis. The minor characteristics that lean toward class 1 (cold, cocacola pee) are outweighed by the key characteristics that go toward class 0 (no diarrhea, no hyperpyrexia, no rigidity). Even with certain class-1-favouring characteristics, the prediction cannot be changed by their impact alone. The explanation from the anchor is of form, [('prostraction < = 0.00', 0.26596539641165823), ('diarrhea < = 0.00', −0.17847335185600147), ('0.00 < cold < = 1.00', 0.12170766253274329), ('0.00 < cocacola_urine < = 1.00', 0.10188616860107497), ('rigour < = 0.00', −0.10138009672809778), ('hyperpyrexia < = 0.00', 0.10061178199340995), ('headace < = 0.00', −0.07098058615255148), ('0.00 < Convulsion < = 1.00', −0.05213895393693386), ('fatigue < = 0.00', −0.030286893070520314), ('0.00 < Anemia < = 1.00', −0.02367864099367406)]. Here, 'prostraction < = 0.00', 0.26596539641165823 in the explanation refers to the weight 0.26596539641165823, which pushes the prediction towards class 1. In the same way 'diarrhea < = 0.00', −0.17847335185600147 in the explanation refers to the weight −0.17847335185600147, which pushes the prediction towards class 0. The medical expert receives the information regarding the probable cases and the related information from XAI, which improves the interpretations on the receiving end.

3.2.2. Level-2 diagnosis for confirmation of malaria based on blood smear images

Level-2 diagnosis deals with confirmation of the probable cases found in the level-1. In Level-2, the blood smear images of the probable cases are recieved as input from the healthcare professional. The blood smear

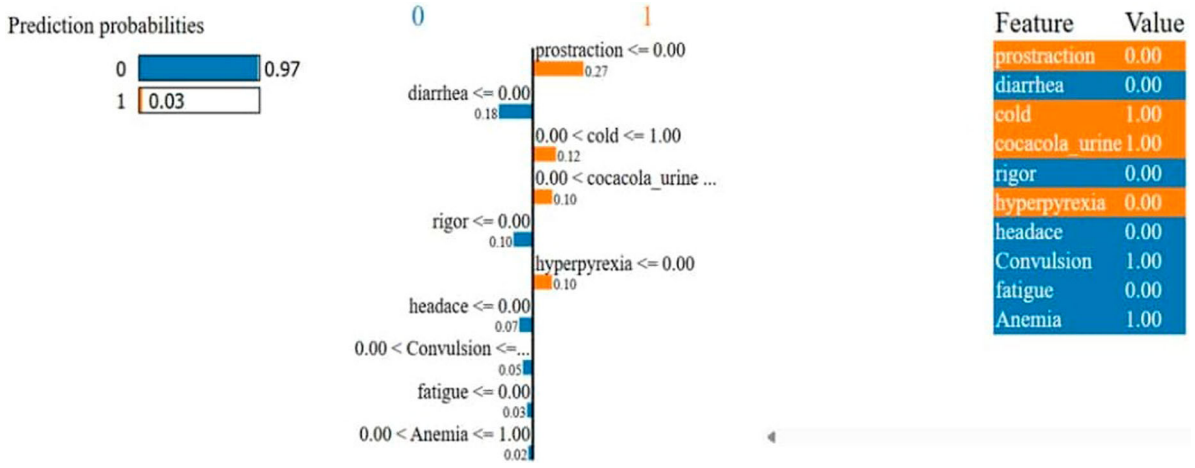


Figure 5. Explanation for a model's prediction using Anchor.

is used as input for the trained model after the suspected cell has been separated from it. The infection status of the input image is determined by the model.

3.2.2.1. Various models. Various models have been developed and analyzed in our study for Level-2 diagnosis of confirmed cases. It includes transfer learning models, vision transformer model and lightweight models.

3.2.2.1.1. Transfer learning models. Knowledge acquired from training a model on one task can be applied to another, unrelated but similar task using a process called transfer learning (Z. A. Khan et al., 2025). In neural network contexts, 'transfer learning' describes the use of models previously trained on huge data sets for tasks like image recognition. Refinement of learnt features and representations of the pretrained models on a new, smaller dataset or for a different task is the primary objective of transfer learning (Assaduzzaman et al., 2025). The comparison of several models that have been examined using the malaria cell image dataset is provided in Table 3. The whole dataset has been divided in the ratio 75:25, where 75% of the dataset images were used for training and the rest 25% for testing. VGG16, VGG19, Vision Transformer and Efficient net are all transfer learning models. But it is possible to develop them from scratch also. After loading them with pretrained weights (like as from ImageNet) and by freezing the base layers, fine-tuning of the dataset is done. In our work, performance of the various transfer learning models was compared along with the proposed model.

- VGG16

The architecture of VGG-16 has been shown below in Figure 6(A). An image of size 224 × 224 is fed into the conv1 layer as input. After that, a sequence of convolutional layers with a 3 × 3 receptive field process the input image. The convolution stride is represented by a single pixel. For spatial pooling (down sampling), five max-pooling layers are selected, each with stride equal to 2. Some convolution layers are followed by max-pooling layers, which operate over a 2 × 2 pixel window. The set of convolutional layers is followed by three fully connected (fc) layers, each of which has a channel size of 4096, 4096, and 1000. Every neuron in the fc layer receives information from the activations of the neurons in the preceding layer. The ImageNet Large-Scale Visual Recognition Challenge's (ILSVRC) category count is indicated by the size of 1000. The soft-max layer comes last. The rectification (ReLU) non-linearity layer equips all of the hidden layers. The architecture's main benefit is its good generalization to other datasets (Kaur & Gandhi, 2019, December).

- VGG19

VGG19 is a 19-layer DCNN with six primary structures, each of which typically consists of fully linked layers and numerous connected convolutional layers. The input is 224 × 224 × 3, and the convolutional kernel is 3 × 3. Pictures can be classified into 1000 object categories by the pretrained network. In Figure 6, the details about

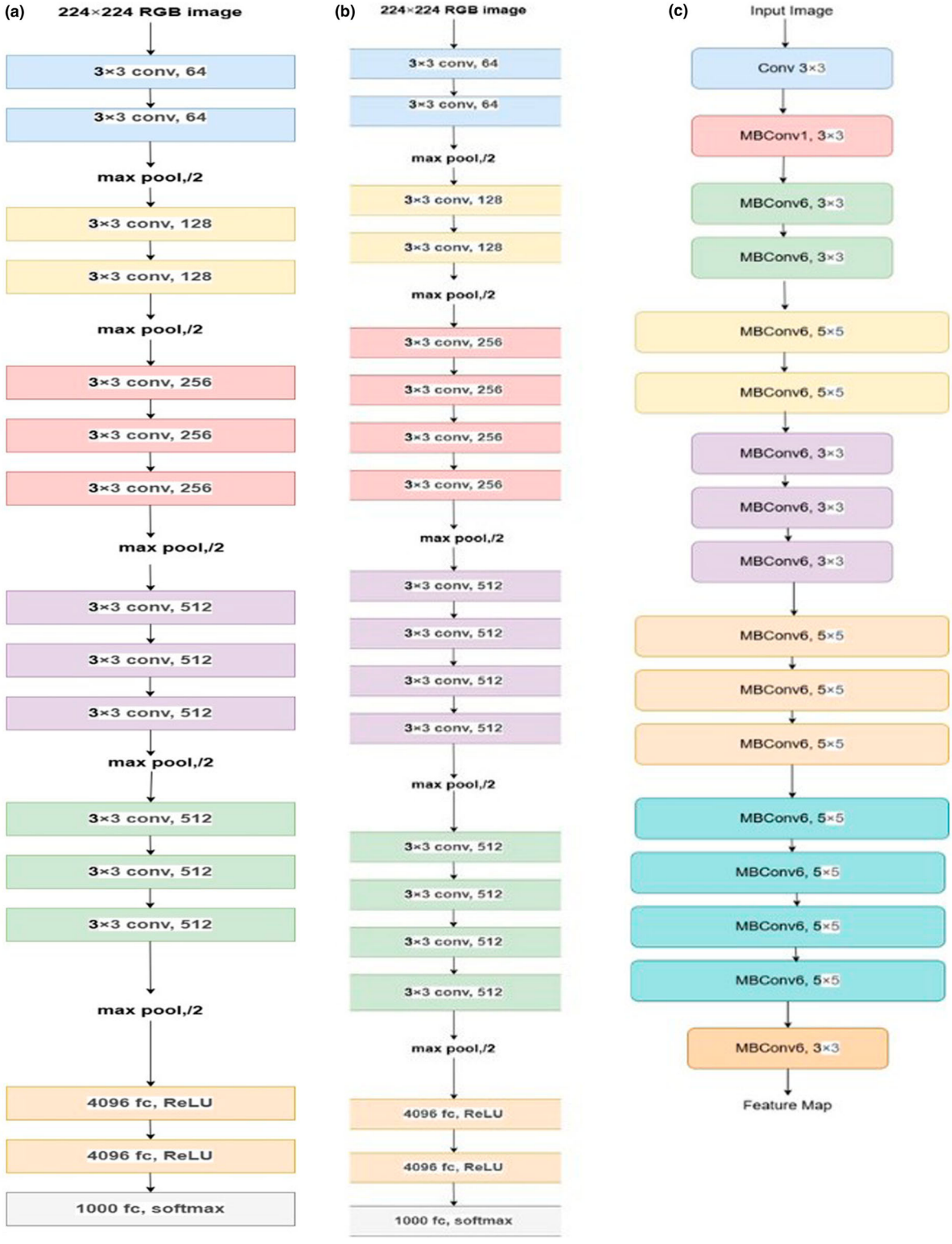


Figure 6. Architecture of (A) VGG16 (B) VGG19 and (C) EfficientNet.

the network are displayed. With all of the convolutional layers present, it may produce incredibly accurate classifications while having a very basic structure with a straightforward linear chain of layers (Shiuly et al., 2022). The architecture of VGG-19 has been shown above in Figure 6(B).

- EfficientNet

Google AI created the EfficientNet family of convolutional neural networks, which are incredibly accurate and computationally efficient. The core idea of EfficientNet is compound scaling, which scales both depth, breadth, and resolution dimensions equally using a compound coefficient. A fixed resource budget is typically used to create Convolutional Neural Networks (ConvNets), which can then be scaled up for increased accuracy with additional resources. After carefully examining model scaling, better performance can be obtained by carefully balancing network depth, width, and resolution. These findings led to the development of a novel scaling technique that use a simple yet remarkably potent compound coefficient to uniformly scale all resolution, depth, and breadth dimensions. Additionally, a new baseline network has been designed and scaled up using a neural architecture search to produce a family of models known as EfficientNets that outperform earlier ConvNets in terms of accuracy and efficiency (Tan & Le, 2019, May). Compound scaling is used by EfficientNet to scale up from EfficientNet-B0, the basic network, to EfficientNet-B1 to B7. EfficientNet and EfficientNetB0 models has been evaluated in our work. The architecture of EfficientNet has been shown in Figure 6(C).

3.2.2.1.2. Vision Transformer (ViT) Models. Dosovitskiy et al. found the ViT concept in (Dosovitskiy et al., 2020) following the success of Transformers in resolving issues related to natural language processing. ViT uses about four times less computational power than state-of-the-art CNN when properly trained on data. ViT makes every effort to mimic the original transformer architecture. Splitting the input image into fixed-size patches and flattening each into a 1D vector is the first step in the procedure. To maintain spatial information, positional embeddings are added after these vectors have been projected linearly into a shared embedding space. The sequence is prefixed with a unique categorization token (patch 0), and a Transformer encoder receives the concatenated embeddings. In order to capture global interactions between patches, the encoder makes use of self-attention (Xing et al., 2022, March; Mzoughi et al., 2024). After receiving the output corresponding to the classification token, a Multi-Layer Perceptron (MLP) head produces the final classification output, which may or may not be infected. In our work, we have analysed both pretrained vision transformer and vision transformer from scratch for our dataset. The general architecture of vision transformer is depicted in Figure 7 below. Figure 8 depicts how results are attained using the pretrained vision transformer model for malaria diagnosis. The prediction results from pretrained ViT is depicted in Figure 8. In Figure 8, the prediction result made by the ViT model is uninfected with probability 0.992 for the Figure 8(A) and for the Figure 8(B), the prediction result made by the ViT model is Parasitized with the probability 0.99. The probability describes the credibility of the prediction made by the model.

3.2.2.1.3. Lightweight Models. Lightweight network models can be used in situations where computing resources are scarce because of their excellent accuracy, low parameter counts, and low computational costs (Fang et al., 2023). Convolutional neural networks have limitless potential for development due to the ongoing advancements in deep learning theory and computer hardware. In the study Fang et al. (2023), the effectiveness of a lightweight multiscale CNN model for identifying and categorizing six wheat illnesses have been assessed. To increase the efficiency, the attention mechanisms are added to the residual structures and the Inception modules are merged with the residual modules. Multiscale-CNN has been proposed in paper (Batool & Byun, 2025). According to Batool and Byun (2025), the Multiscale-CNN continuously performs better than the cutting-edge DL architectures, including Inception V3, ResNet, and AlexNet. According to the empirical investigation, the suggested M-CNN performed better than expected and demonstrated its ability to correctly categorize brain tumours while lowering the danger of overfitting. The study emphasizes how important feature selection is for improving overall classifier performance and computing efficiency. Furthermore, the suggested M-CNN architecture yields reductions in parameter size of roughly 98%, 93%, and 91% in comparison to Inception V3, ResNet-50, and AlexNet respectively making it noticeably lighter than the current architectures. Lightweight DenseNet (LWDN), a novel lightweight deep convolutional neural network, has been proposed in paper (Dheeraj & Chand, 2024) for the detection of plant leaf disease in agricultural applications. The proposed DenseNet121 architecture has concatenated and pruned architecture of DenseNet121. This work demonstrated the creation of a reliable and accurate liver segmentation technique. A very light convolutional neural network (Ga-CNN) with randomly initialized Gaussian weights was used in the developed algorithm to segment the data. It did this by inserting the data steadily and drawing connections between the magnitudes of the features our network produced while maintaining the metric evidence of the data's complexity and the original manifold. Each convolutional layer is followed by a ReLU activation function, which speeds up learning and detects early convergence. Additionally, by implementing local response normalization (LRN) following ReLU, the false positive rate has been reduced and precision has been increased

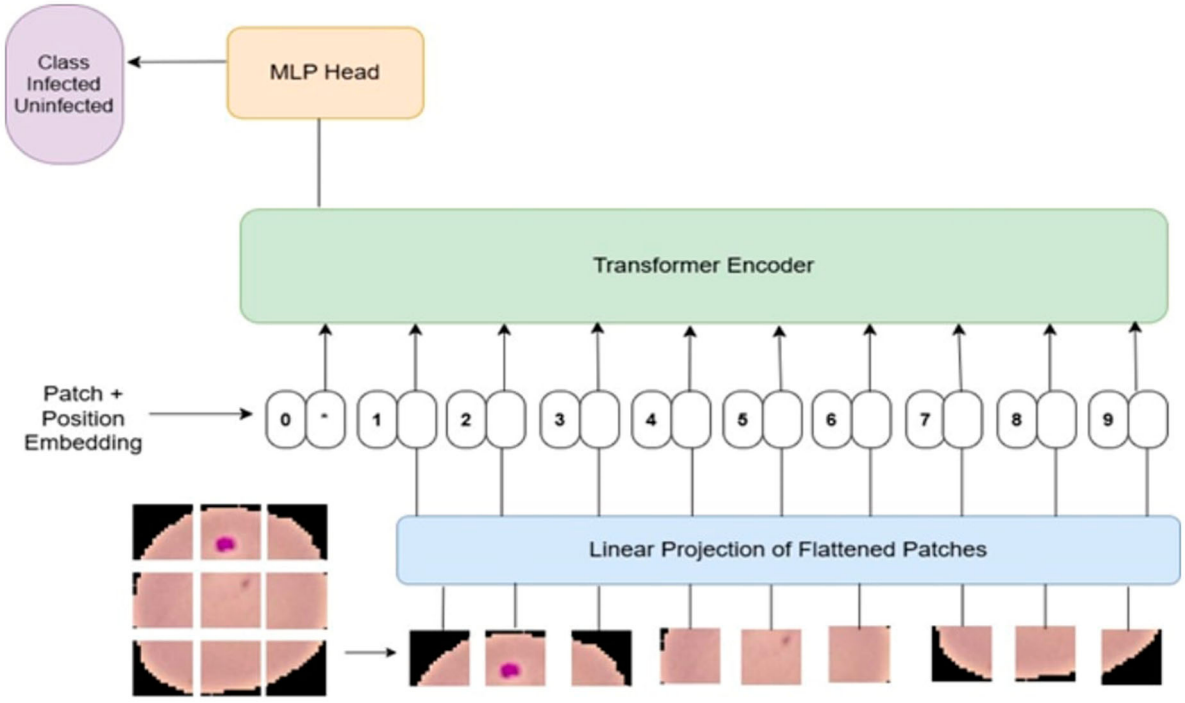


Figure 7. Architecture of Vision Transformer (ViT).

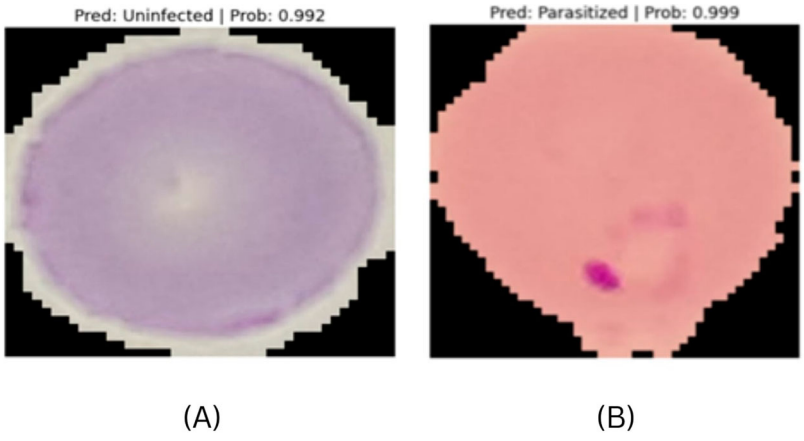


Figure 8. Predictions by pretrained Vision Transformer (ViT) (A) Uninfected cell image (B) Parasitized cell image.

Table 6. Comparison between Transfer Learning models, Lightweight CNN models and Deep Neural Network models.

Feature	Transfer-Learning Models	Lightweight CNN Models	Deep Neural Network Models
Size	Medium to Large	Small	Varies
Computation cost	Medium	Low	High (if large)
Used For	Small dataset	Resource constrained environment	Large Dataset
Example Models	VGG16, VGG19	Lightweight custom CNNs	Fully connected, custom CNN
Computation Speed	Slower unless using GPU	Fast	Slow
Model Size	Very Large (20MB–550MB)	Small (1–5 MB)	Large($>$ 20MB)
Memory Usage	Very High	Low to Moderate	High

(Ahmad et al., 2022). The EfficientNetV2-Small model performed exceptionally well in identifying sunflower illness (Zhang & Wu, 2025). SqueezeNet1.1 model has been used for the classification of malaria in the resource constrained settings with fewer parameters (Nettur et al., 2025). A general comparison has been shown below in Table 6.

3.2.2.2. Proposed model for level-2 diagnosis. A Lightweight CNN model has been proposed. Following are the methodology followed to develop the proposed model. The steps carried out for training model for level-2 diagnosis is depicted in Figure 9.

3.2.2.2.1. Data partition. The dataset used is 'Malaria Cell Image Dataset' from Kaggle. This dataset is actually taken from NIH (National Institute of Health) database. The entire dataset has been divided in to 75:25 ratio. The model was trained using 75% of the images, then it was tested using the remaining 25%.

3.2.2.2.2. Data preprocessing. Two instances of ImageDataGenerator are created. One for test data that only uses rescaling, and another for training data that has many augmentations (rotation, shifts, shear, zoom, flips, and rescaling) to increase model robustness. The generators create batches of data that are ready for the model by loading photos from the train_path and test_path directories, respectively.

3.2.2.2.3. Proposed lightweight CNN model. An input layer and three convolutional layers (32, 64, and 128 filters) with batch normalization and ReLU activation constitute the model. After the first two convolutional layers come the max pooling layers. Prior to a thick layer (256 units) with L2 regularization and dropout (0.3), a global average pooling layer lowers dimensionality. A softmax output layer for class probabilities is the last layer. The model is built using the Adam optimizer (learning rate 0.01the categorical crossentropy loss function, and two metrics: accuracy and top-5 categorical accuracy) for multi-class classification. In order to enhance training, three callbacks are defined. If validation loss does not improve after 10 epochs, EarlyStopping terminates training early; If validation loss reaches a plateau, ReduceLROnPlateau lowers the learning rate; The optimal model is saved by ModelCheckpoint according to validation accuracy. A customized callback GradientAccumulation simulates a bigger batch size by accumulating gradients over several batches prior to performing an update. This is useful when huge batch sizes are not possible due to limited memory. The generators supply the training and validation data, and the model is trained using.fit(). The size of the data determines how many steps there are in each epoch. To control training dynamics, callbacks including gradient accumulation are passed. The layers in the proposed model is depicted in Figure 10.

According to Figure 9, a 224 × 224 × 3 (RGB) input image is fed into the model, which then runs it through several convolutional and pooling layers. Layers of Conv2D + Batch Normalization stabilize training while extracting hierarchical features. Only significant features are retained as the spatial dimensions are gradually reduced using MaxPooling2D layers. The feature maps are reduced to a vector of length 128 using a Global Average Pooling layer. Lastly, classification is done using Dense + Dropout + Dense layers, which produce probabilities for two classes. Layerwise description is as follows. Conv2D is the first convolutional layer that generates 8,968 parameters and has 32 filters. It is followed by BatchNormalization that uses 128 parameters to normalize the output. MaxPooling2D is the next layer that uses 0 parameters to reduce spatial dimensions to (112, 112, 32). It is followed by Conv2D layer which is the second convolutional layer that generates 18,496 parameters by using 64 filters. It is later followed by BatchNormalization layer that uses 256 parameters to normalize. The next layer is MaxPooling2D layer which uses 0 parameters to reduce dimensions to (56, 56, 64). It is then followed by Conv2D layer which produces 73,856 parameters as a third convolutional layer, which has 128 filters. It is then followed by BatchNormalization layer which uses 512 parameters to normalize. It is followed by GlobalAveragePooling2D layer which uses 0 parameters to reduce spatial dimensions to (128). It is then followed by the Dense layer which has 256 units in a fully connected layer that generates 33,024 parameters. It is then followed by a Dropout layer which produces 0 parameters by applying dropout at a rate of 0.256. It is then followed by a Dense layer which is a two-unit final output layer that generates 514 parameters. As the network moves from input to output, it processes and categorizes the data using convolution, normalization, pooling, and dense layers.

3.2.2.3. Testing the proposed CNN model. The input to the model is usually blood smear images which is collection of cells along with or without the infected cell.

3.2.2.3.1. Segmentation of the suspected cell from blood smear image. The segmentation is carried out with the help of image processing techniques as shown in Figure 11. Isolation and extraction of a circular crop of a suspected cell on a white background is carried out by processing a malaria blood smear image. To make colour-based segmentation easier, it first reads and displays the original image before converting it to the HSV (Hue-Saturation-Value) colour space. Based on hue, saturation, and value thresholds, a binary mask is created to emphasize contaminated regions, which are usually tinted blue or purple (Najjar et al., 2023, February). The mask is cleaned using morphological processes, which join broken areas, fill in gaps, and eliminate noise. After

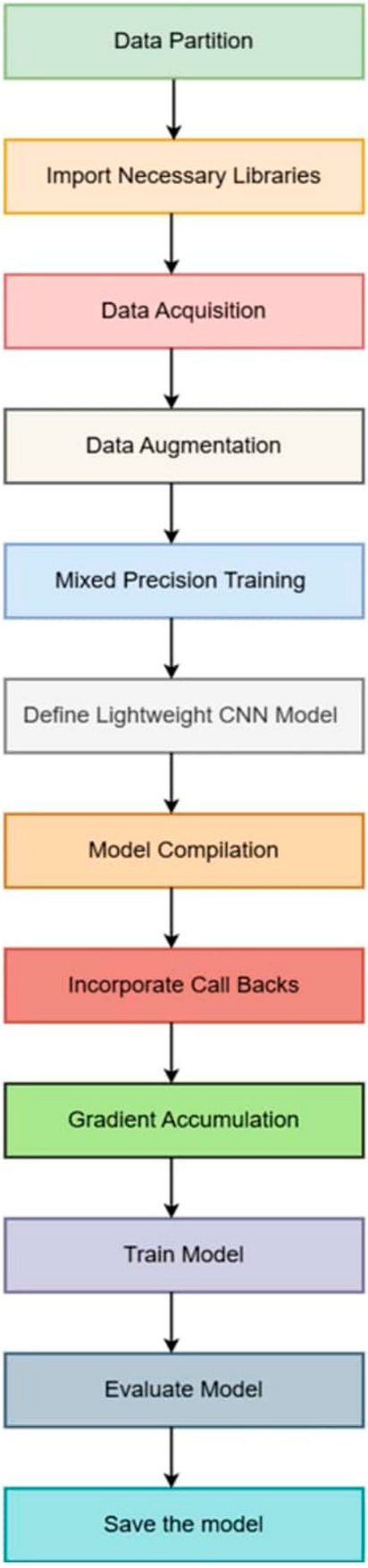


Figure 9. Basic steps in training the model for Level-2 diagnosis.

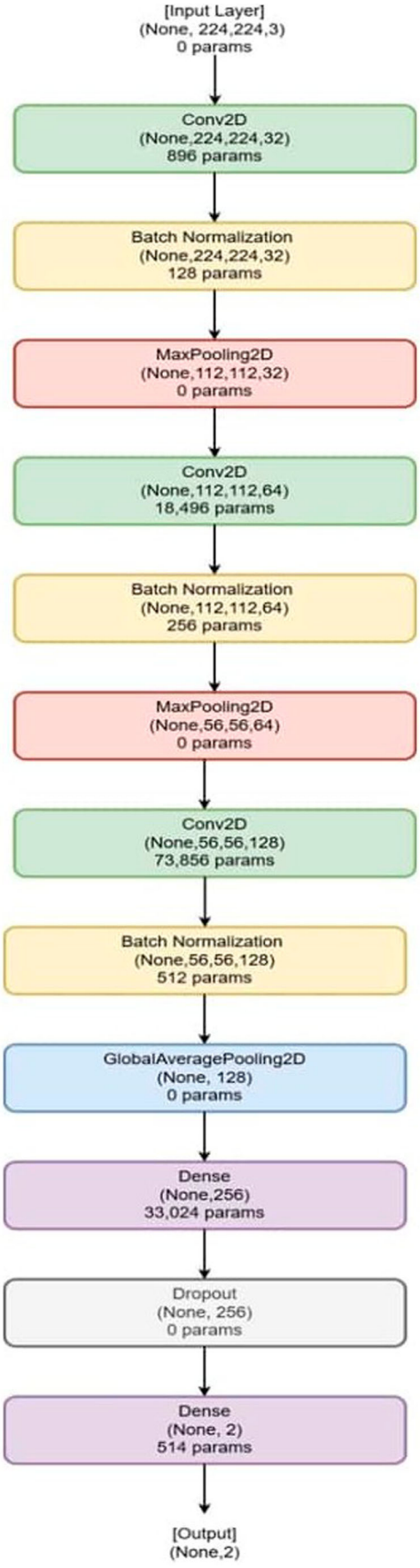


Figure 10. Layers in the proposed lightweight CNN model for Level-2 diagnosis.

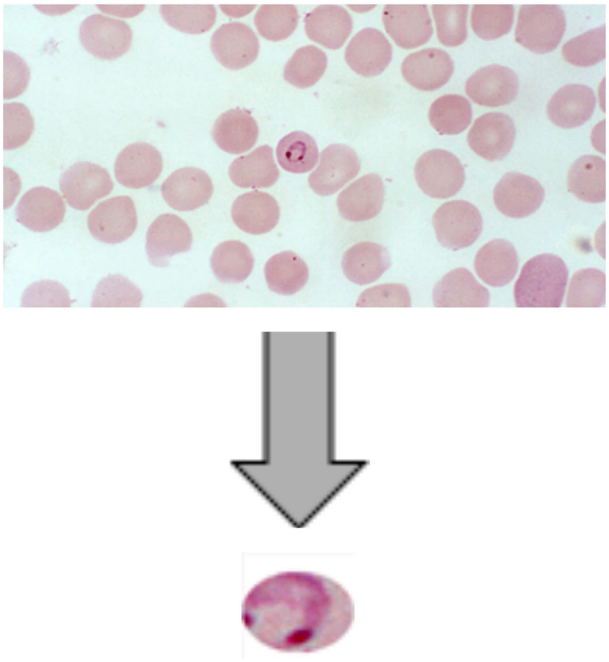


Figure 11. Blood smear image along with the segmented suspected cell image.

Algorithm 1: Algorithm for segmentation of suspected cell image from the blood smear image

Input: An RGB image which is a blood smear image containing group of cells
Step 1: Load the input image
Step 2: Convert the RGB image to HSV
Step 3: Create binary mask for infected area
Step 4: Clean the mask by removing noise, filling holes and apply morphological closing to connect nearby regions
Step 5: Label connected regions in the mask, measure bounding box and area of each region
Step 6: If no region is found, stop and raise error
Step 7: Identify the largest area and extract its bounding box
Step 8: Add a fixed margin to the bounding box in all directions and clip it to stay within image dimensions
Step 9: Crop the cell region from the original image using the expanding bounding box
Step 10: Create circular mask
Step 11: Create white background image of same size
Step 12: Replace the circular mask area in white background with corresponding pixels from the cropped cell.
Step 13: Display final result
Output: A circularly cropped image of the suspected cell placed on a white background

locating related elements in the mask, the code chooses the largest area, which is thought to be the primary infected cell and enlarges its bounding box to guarantee complete coverage. A circular mask is made and applied, setting any non-circular parts to white, once this portion of the original image has been cropped. The circularly clipped cell is then shown, providing a clear and concentrated image perfect for further study. Figure 11 showcases a suspected cell image which has been segmented from the blood smear image. The steps carried out for the process is explained below in Algorithm 1. The algorithm has an O(M × N) time complexity, where M × N is the total number of pixels in the input image. Pixel-wise operations over the full image in steps 1–5 and operations over the cropped area in steps 9–13 dominates the algorithm.

3.2.2.3.2. Input to the proposed model. The segmented cell image is given as input to the trained model. The result is either infected or uninfected. The final prediction information is forwarded to the cloud for long-term storage as well as to the healthcare professional.

3.2.2.4. Performance evaluation parameters. Specificity, F1-score, recall, accuracy, and precision are employed to assess and contrast the effectiveness of the suggested strategy.

True Positives (TP): The classifier and the target class both anticipate a favourable result.

True Negatives (TN): As the classifier predicted, the target class is also negative.

False Positives (FP): When the target class is positive but the classifier predicts it to be negative.

False Negatives (FN): When the classifier predicts a target class to be negative when it is positive.

Table 7. Comparison between GradCAM and GradCAM++.

Method	Speed	Complexity	Steps	Suitability
Grad-CAM	Fastest	Low	One backward pass (first order gradients)	Best for edge and fog devices
Grad-CAM++	Moderate	Medium	Requires 2nd and 3rd order gradients	Suitable for fog nodes with GPU

3.2.2.4.1. Accuracy. Accuracy is one performance indicator that may be employed to calculate the proportion of accurately categorized predictions (Bader et al., 2024; Pandey et al., 2024). Equation (6) is used to express it.

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}} \tag{6}$$

3.2.2.4.2. Precision. The ratio of correctly detected positive efforts to all positive predictions is known as precision (Bader et al., 2024; Pandey et al., 2024). Precision can be described using equation (7).

$$\text{Precision} = \frac{\text{True Positives(TP)}}{\text{True Positives(TP)} + \text{False Positives(FP)}} \tag{7}$$

3.2.2.4.3. Recall. The percentage of positive expected outcomes to all predictions in a class is determined using recall (Bader et al., 2024; Pandey et al., 2024). Recall can be described by Equation (8).

$$\text{Recall} = \frac{\text{True Positives(TP)}}{\text{True Positives(TP)} + \text{False Negatives(FN)}} \tag{8}$$

3.2.2.4.4. F1 score. The F1 Score is the harmonic mean of precision and recall (Bader et al., 2024; Pandey et al., 2024). Equation (9) can be employed to describe the F1 Score.

$$\text{F1 Score} = 2.\frac{\text{Precision.Recall}}{\text{Precision} + \text{Recall}} \tag{9}$$

3.2.2.4.5. Specificity. The percentage of real negatives among all actual negatives is known as specificity, or the true negative rate (Bader et al., 2024). Additionally, the Specificity can be described using Equation (10).

$$\text{Specificity} = \frac{\text{True Positives(TP)}}{\text{True Negatives(TN)} + \text{False Positives(FP)}} \tag{10}$$

3.2.2.4.6. Receiver operating characteristic (ROC-curve). A graphical representation of the performance of the binary classification system when its discrimination threshold is altered is called a ROC curve. It is frequently used to assess the performance of probability-based models. The True Positive Rate (TPR) is represented by the Y-axis of the curve, while the False Positive Rate (FPR) is shown by the X-axis. Every ROC curve has a corresponding diagonal line. When the ROC curve is above this diagonal line, the model performs better than random guessing. Otherwise, the model could be improved and performs worse than random.

3.2.2.5. Explainability using GRADCAM. Different XAI techniques like GradCAM, GradCAM++, Score-CAM, Layer-CAM, Ablation CAM have been analysed. Among these techniques, only two techniques which support fog computing environments were shortlisted. GradCAM and GradCAM++ are those techniques. A comparison between these techniques has been done (Chattopadhay et al., 2018; Selvaraju et al., 2017, March; Lerma & Lucas, 2022). Table 7 shows the comparison between GradCAM and GradCAM++.

The region that the model takes into account when achieving a decision is illustrated and analyzed using the XAI technique GradCAM. The fundamental idea behind the GradCAM is to assign significance scores to each neuron for a particular importance choice using the gradient information that enters the CNN's final convolutional layer (Ahmed & Imran, 2024; Selvaraju et al., 2017). A generalization of the CAM model, the Grad-CAM framework caters to the requirements of numerous end users. It creates saliency maps that highlight the salient characteristics of the target image based on its class. Importance of each feature maps for a target class in the last convolutional layer is evaluated by Grad-CAM. In the final convolutional layer, it computes the gradients of the target class score with respect to the feature association. These gradients illustrate the impact of feature map adjustments on the target class score (S. Kaushik et al., 2024; Lalithadevi & Krishnaveni, 2024; Mzoughi

et al., 2024, December). Without changing the model architecture or requiring retraining, the Grad-CAM produces visual explanations. Once the gradient of any target class c is known, the activation feature map M of a single layer l is globally averaged over the width, height, and depth dimensions. The GCAM class discriminative heat map is produced by combining these activation maps in a weighted manner, and the ReLU function is then applied. Equations 11 and 12 use α_l^C to describe the neuron significance weights.

$$GRADCAM_l^C = ReLU\left(\sum_l \alpha_l^C M^l\right) \tag{11}$$

Here in equation (11), *c* denotes the class and *l* denotes the convolutional layer.

M^l *is* the feature map at convolutional layer *l* which is the output of convolution filters which is of size $H \times W$. *Here* α_l^C is importance weight of feature map M^l *for class c. It is* computed as global average pooling of gradients of the score for class *C* (y^c) with respect to M^l which is denoted using equation (11).

$$\alpha_l^C = \frac{1}{N}\sum_x\sum_y\sum_z \frac{\partial y^c}{\partial A_{x,y,z}^l} \tag{12}$$

In equation 12, y^c means the score of the network for class *c*. $A_{x,y,z}^l$ means the activation position (*x,y,z*) in the feature map of layer *l. x,y* are the spatial co-ordinates (height & width). *Z* is channel or depth index inside that feature map (if the layer output is 3D). $\frac{\partial y^c}{\partial A_{x,y,z}^l}$ is the gradient of the class score with respect to that activation. It measures the sensitivity of the prediction for class *c* is to that of unit's activation. $\sum_x\sum_y\sum_z \frac{\partial y^c}{\partial A_{x,y,z}^l}$ denotes sum over all spatial along with depth positions of the feature map. N is the normalization factor which is the total number of units in the feature map. (i.e. $N = H \times W \times D$). The Rectified Linear Unit (ReLU) function in equation 11 is used to make sure that only positive influences are retained that is, only features that promote class *c* are kept and not those that repress it. After they are acquired, the Grad-CAMs are scaled to match the dimension of the input volume. These heat map-like Grad-CAM slices are then superimposed on corresponding input slices of different sizes. The output heat map that is created shows the locations in the input slice that the trained model utilizes to provide predictions. The diagram helps identify the crucial areas that influence the predictions of the model and enhances comprehension of the decision-making process (Mzoughi et al., 2024). Grad-CAM (Gradient-weighted Class Activation Mapping) is a CNN visualization technique that uses gradients flowing into the final convolutional layer to highlight image regions that are particularly important for a certain class prediction. It is model-agnostic for CNNs, meaning it can work with almost any CNN architecture without requiring modifications or retraining, and it generates user-friendly heatmaps that provide for instant visual insight into the spatial areas the model responds to. By globally averaging gradients with respect to feature maps, the approach calculates class-specific importance weights. These weighted maps are then combined, and a ReLU operation is applied to highlight regions that contribute favourably. For ease of reading, the resulting coarse saliency map is upsampled and superimposed on the original image. Empirical research demonstrates that Grad-CAM improves localization accuracy and interpretability over previous approaches like CAM. It also helps users, even inexperienced observers, comprehend and trust model predictions more. It is a strong and popular tool in computer vision because of its wide range of applications in tasks including picture classification, visual question answering, and captioning (Selvaraju et al., 2017). Other XAI techniques like SHAP and Lime are expensive and unstable respectively for image analysis. The GRADCAM results for the input image using our proposed model has been depicted below in Figure 12 infected area has been highlighted.

4. Results and discussion

This study introduces a dual-level framework for the diagnosis of malaria. Lightweight models for resource-constrained environments have been proposed for each level. For level-1 diagnosis, which involves finding the probable cases, a lightweight MLP model has been proposed. In order to identify the confirmed cases for the level-2 diagnosis, a lightweight CNN model has been suggested.

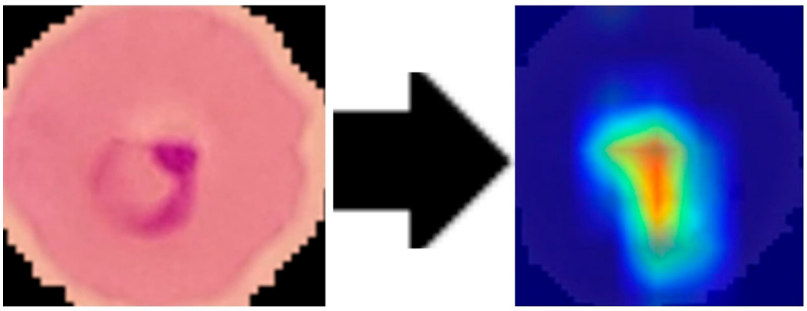


Figure 12. Grad-CAM result for an infected image (A) Input Image (B) Highlighted Output Image from Grad-CAM.

Table 8. Table depicting various performance metrices values of the proposed MLP model for Level-1 diagnosis.

Performance Metrices	Values
Accuracy	0.96
Precision	0.95
F1-Score	0.92
Recall	0.90

Table 9. Comparing the performance of several ML/DL models with the proposed lightweight MLP model for Level-1 diagnosis.

ML and DL models	Accuracy	Precision	Recall	F1-Score	Specificity
Decision Tree	0.55	0.38	0.40	0.39	0.72
Random Forest	0.59	0.33	0.20	0.25	0.83
SVC	0.64	0.33	0.10	0.15	0.92
SVC-gridCV	0.64	0.33	0.10	0.20	0.90
Logistic Regression	0.63	0.40	0.16	0.22	0.88
RNN	0.50	0.53	0.52	0.52	0.48
KNN	0.57	0.29	0.19	0.23	0.76
XGBoost	0.55	0.53	0.55	0.54	0.69
Extra Trees	0.57	0.53	0.57	0.54	0.76
DNN	0.52	0.52	0.52	0.51	0.66
AdaBoost	0.54	0.49	0.54	0.50	0.75
NaiveBayes	0.57	0.63	0.57	0.60	0.59
Proposed MLP Model	0.96	0.95	0.90	0.92	0.98

4.1. Results obtained for level-1 diagnosis

As diagnosis is made in a fog computing environment, it is important to take into account one important aspect. That is, the model developed should be light. At the same time, the efficiency of the model cannot be compromised. It should have the capacity to predict probable cases with high accuracy. It is clear that the suggested MLP model works better and is more accurate than other models.

Table 8 below shows the performance metrics of the proposed lightweight MLP model. Additionally, Table 9 presents an analogy between the proposed one and the other models. Our model is comparatively lighter and can be used in the resource-constrained environment.

Gridsearch optimization is used in developing the level-1 model. {'batch_size':16,'model__dropout_rate': 0.2,'model__units1': 64,'model__units2': 32,'model__units3': 16} are the ideal parameters discovered by Grid-search optimization. The ideal number of neurons for the three dense layers of the suggested MLP model is 64, 32, and 16.

The model is assessed using the test data after training. An H5 file containing the trained model is saved for subsequent use. The test data is used to generate predictions. A classification report is printed that includes a confusion matrix, precision, recall, and F1-score. Specificity and sensitivity are extra metrics that are directly calculated from the confusion matrix if the objective is binary classification, which is what it is in this case. The

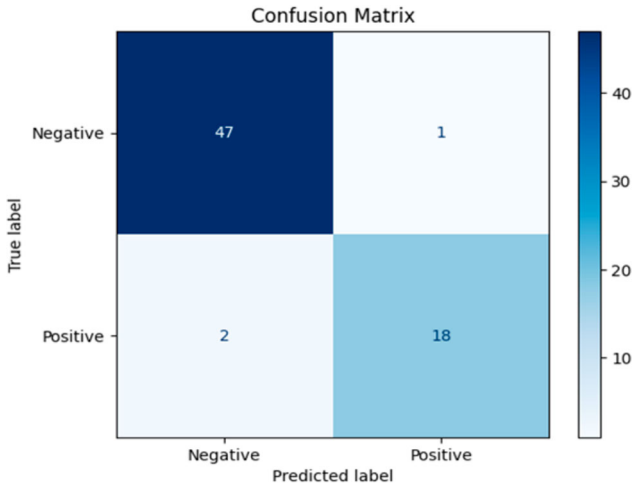


Figure 13. Confusion matrix for the MLP model's performance at Level 1.

Table 10. Comparing the traditional models with the proposed lightweight MLP model based on parameter count and inference time.

ML and DL models (in microseconds)	Parameter count	Inference time per sample
Decision Tree	8191	0.2
Random Forest	1228500	0.29
SVC	4500	0.05 - 0.2
KNN	5155	0..2 - 0.5
Proposed MLP Model	3905	0.05–0.1

ROC curve and AUC (Area Under Curve) for binary classification has been calculated for plotting the ROC to show how well the model can differentiate between the two classes.

The confusion matrix in Figure 13 provides a visual representation of the suggested model's performance on a test dataset. Four results are displayed in the matrix:47 True Negatives (TN), or negative cases (top-left cell), were accurately predicted by the model. One negative instance was erroneously predicted as positive, resulting in a False Positive (FP) (top-right cell). When two positive cases were erroneously predicted as negative, the result was a False Negative (FN) (bottom-left cell). The model correctly predicted 18 True Positives (TP), or positive cases (bottom-right cell). Overall, the model shows strong performance with very few misclassifications. Table 10 compares the performance of the proposed MLP model with the traditional models in terms of parameter count and inference time. Our proposed lightweight MLP model is efficient with a much lesser parameter count and reference time than other traditional models.

Figure 14 shows the proposed MLP model's ROC curve. At different threshold values, the ROC curve displays the trade-off between the false positive rate and the real positive rate (sensitivity). The model's performance is assessed using the AUC (Area Under the Curve) value. Perfect classification is indicated by a score of 1.0, whilst no discriminative ability (random guessing) is shown by a score of 0.5. With a remarkable AUC of 0.99, the test (orange curve) and training (blue curve) ROC curves in this plot demonstrate the model's great discriminative power and good generalization to new data with minimal overfitting. The model performs better the closer the curves are to the upper-left corner.

Above Figure 4 depicts how explainable AI (XAI) increases the predictability of the suggested MLP model's predictions. As MLP models are usually black boxes, it is difficult to classify which attributes are responsible for the positive or negative outcome of the result. Explainable AI makes the process easy by analysing and explaining which features give positive outcomes and which are the features giving negative outcomes.

4.2. Results obtained for level-2 diagnosis

In this level, the healthcare professional feeds the blood smear images of the probable cases. The blood smear images are a collection of cells present in our blood. It may or may not have the infected cell. The suspected cell

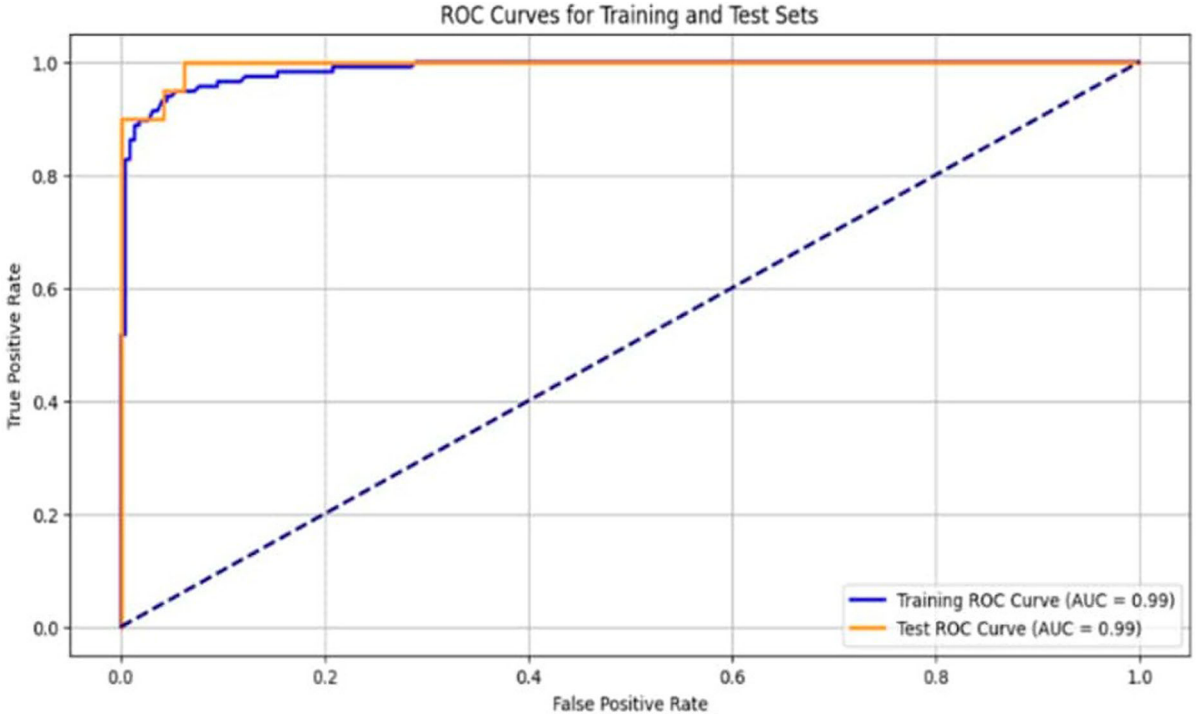
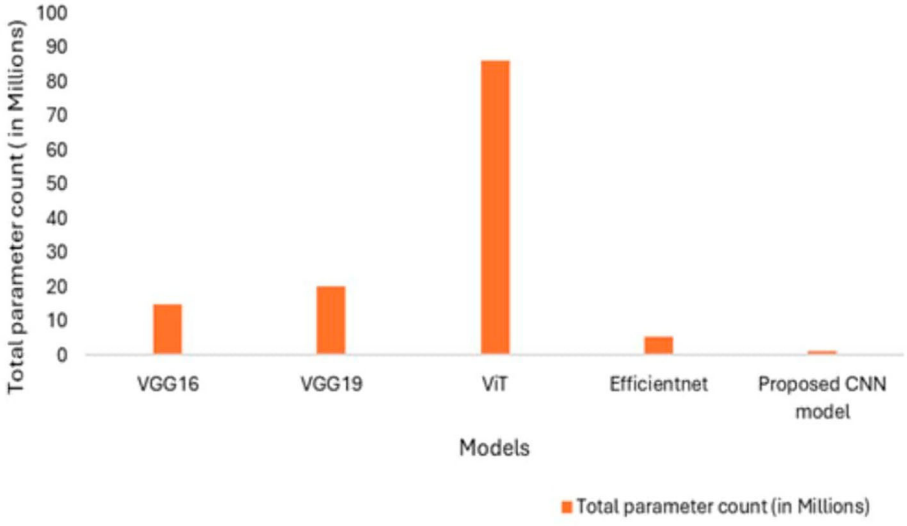


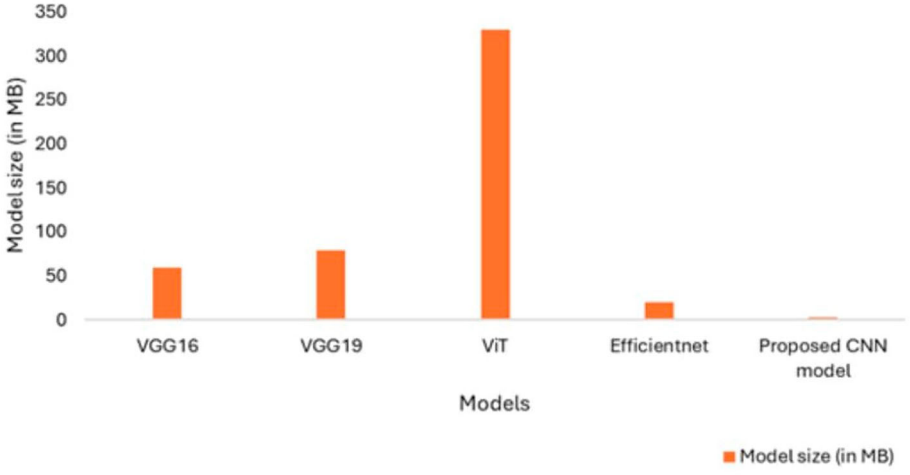
Figure 14. ROC curves for the MLP model.

is segmented from the blood smear image using algorithm 1 shown above. The single-cell image is resized and given as input to the trained CNN model. Which in return classifies the cell as infected or uninfected. Measures are taken to make the model both lighter and more efficient as the prediction process takes place in the fog computing environment. A number of models, including transformer and transfer learning models, have been compared to the suggested CNN model. VGG16, VGG19, ViT, EfficientNet are some of the efficient models compared with our proposed model in Figure 15. Parameter count, model size and accuracy were some of the important factors considered while developing the model for the fog computing environment. Figure 15(A) depicts the variation in the parameter count between different models and our proposed CNN model. The proposed CNN model appears to be a lightweight model because of its extremely low parameter count, which less than 1 million. Because of this, it is better suited for settings with limited resources. The effectiveness of the suggested CNN model in comparison to bigger, more intricate structures is highlighted in the chart. Despite having significantly fewer parameters, such models can still perform well when designed carefully, highlighting the trade-off between model size and performance. The model sizes (in megabytes, MB) of many deep learning architectures, such as VGG16, VGG19, ViT (Vision Transformer), EfficientNet, and a proposed CNN model, are compared in the bar chart in Figure 15(B). The names of the models are shown on the X-axis, while their sizes in megabytes are shown on the Y-axis. Despite having significantly fewer parameters, lightweight CNN models can still perform well when designed carefully, highlighting the trade-off between model size and performance. Additionally, the bar graph that contrasts the accuracy of several models with the proposed CNN model is shown in Figure 15(C). These bar graphs indicates that the proposed CNN model is quite effective at achieving a good balance between computational/resource efficiency and performance.

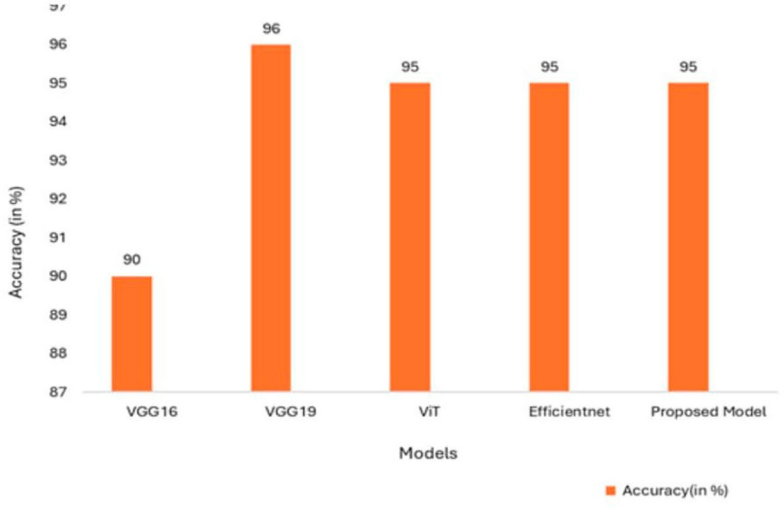
Figure 16 shows the confusion matrix used to assess the suggested CNN model's performance. The confusion matrix evaluates the model's performance by contrasting the expected and actual labels. There are four values in the matrix: 3359 true positives (the model correctly predicted the positive class), 3197 true negatives (the model correctly predicted the negative class), 106 false negatives (the model predicted a negative label when the actual label was positive), and 248 false positives (the model predicted a positive label when the actual label was negative). This allows for the derivation of many metrics for performance. With an overall accuracy of almost 95%, the model was able to accurately classify the vast majority of the cases. With a precision of roughly 93.13%, it is generally accurate when predicting the positive class. At roughly 96.94%, the recall



(A)



(B)



(C)

Figure 15. Bar graphs showing (A) Total parameter count (B) model size (C) accuracy of various models with the proposed CNN model.

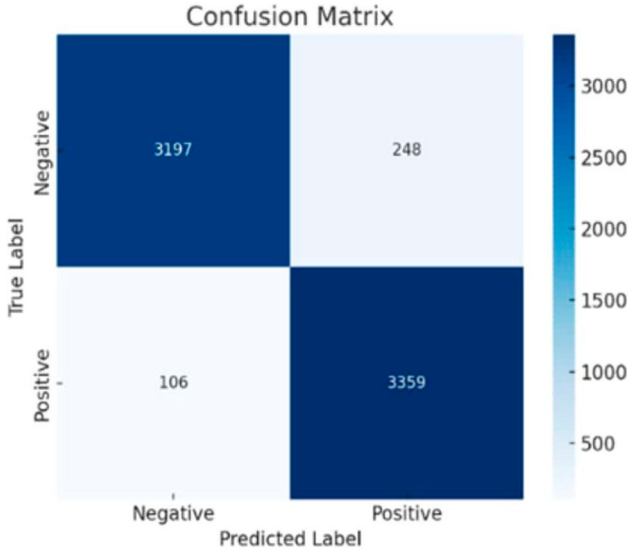


Figure 16. Confusion matrix for the lightweight CNN model.

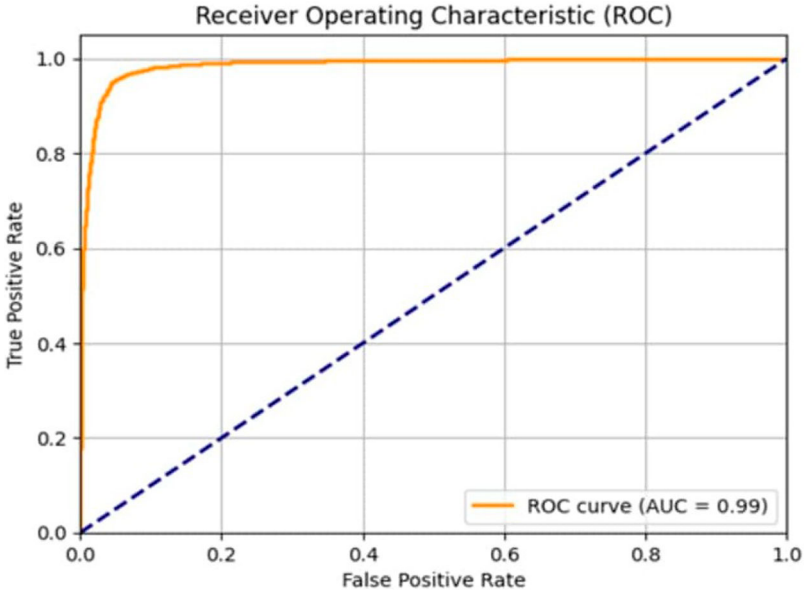


Figure 17. ROC curve for the CNN model.

is even higher, demonstrating how well the model detects true positives. The model maintains a good trade-off between recognizing positive cases and minimizing false alarms, as evidenced by the F1 score of roughly 95.00%, which strikes a balance between precision and recall. The model performs well overall, with minimal rates of misclassification.

Further, the ROC curve is plotted to investigate the performance of the model. The performance of the proposed CNN model is shown as ROC curve in Figure 17. The ROC curve in this instance is orange and closely follows the upper-left corner, indicating greater classification performance. The performance of a classifier is shown by the diagonal dashed blue line (AUC $=$ 0.5). With a value of 0.99, the Area Under the Curve (AUC) is quite near to 1. This high AUC value shows that the model is very good at making accurate predictions with few errors since it can distinguish between the positive and negative classifications. Overall, the ROC curve and its AUC score confirm that the model demonstrates outstanding classification performance.

Prediction and confidence analysis for the trained model has been carried out. The proposed lightweight MLP and CNN models are tested using user input. For the MLP model, the input is in the form of array [1,0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1, 1] that holds o's and 1's which differs based on the feature. Finally, it is classified

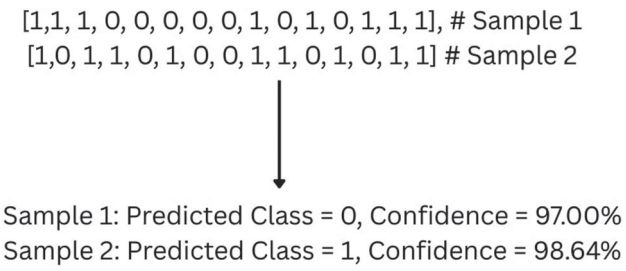


Figure 18. Confidence Analysis on predictions made by the Proposed MLP model.

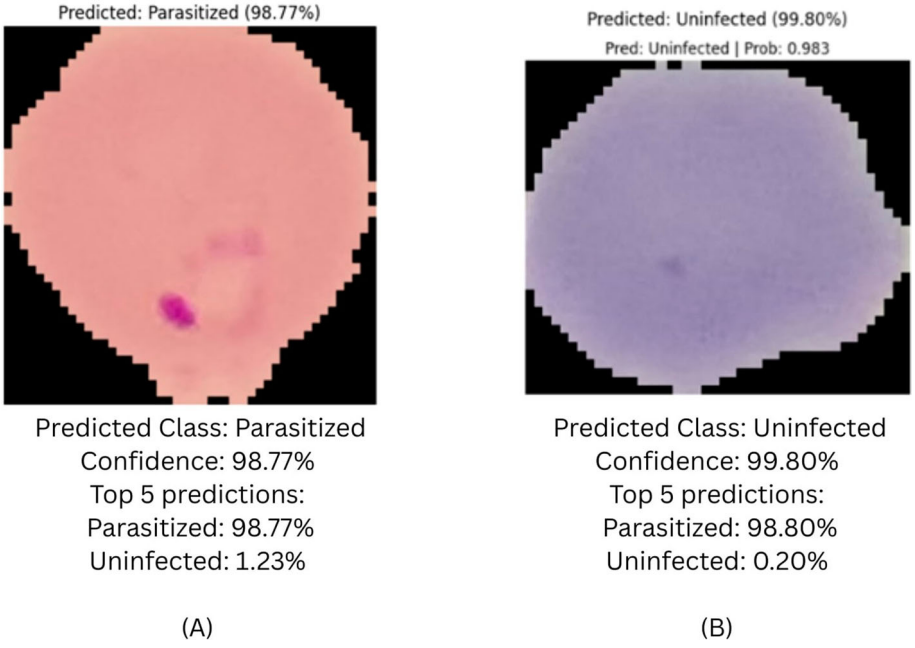


Figure 19. Confidence Analysis on predictions made by the Proposed CNN model for (A) Parasitized image (B) Uninfected image.

as 0 or 1, which is positive or negative and displayed with percentage of confidence. Figure 18 depicts how prediction and confidence analysis is done for the proposed lightweight MLP model. In Figure 15, the input is in the form of samples with 0's and 1's which specifies those 15 features which is given as input to the model for predictions.

For validating the proposed CNN model, the input image is loaded and preprocessed to ensure that it matches the desired input shape of the model. Additionally, a prediction function is provided that loads the CNN model that has been trained, resolves any mixed precision concerns by converting layers to float32, and then uses the preprocessed input image to produce predictions. The function computes the confidence score of the model, extracts the class with the highest probability as the projected label, and makes sure the model's output is a valid probability distribution. Figure 19 given below depicts the confidence analysis for the user input. The confidence percentage is analysed for each predictions.

4.3. Results obtained from simulation of level-1 diagnosis

Simulation of the fog setup for level-1 diagnosis is done using a tool iFogSim (Gupta et al., 2017). This application uses the iFogSim and CloudSim frameworks to simulate fog computing. It simulates level-1 diagnosis of our disease prediction system that has been explainability improved with Anchor XAI. With a cloud server, various fog servers, and numerous edge computers (manual input terminals), the simulation builds a hierarchy of

devices. The fog servers serve as intermediary processing units and house the Anchor explanation (XAI module) and the MLP predictor (disease prediction model), while the cloud, which is at the top level, is utilized to store patient records. Physicians are given edge computers to use for input and output chores. These computers execute manual_input modules and the result_display. Below is the code-snippet in iFogSim that simulates the level-1 diagnosis in our framework. In the simulation 8 edge devices are connected to 3 fog servers. The code in IfogSim along with the simulation is shown below in Figure 20.

The outcome demonstrates how a disease prediction application was successfully implemented in a Fog–Cloud environment that was integrated with Anchor XAI for explainability. Cloud storage, several fog servers, and end-user PCs are among the devices that are initialized at the start of the simulation. Each device

(a)

```
static List<FogDevice> fogDevices = new ArrayList<>();
static int numComputers = 8; // computers (manual input terminals)
static int numFogServers = 3; // fog servers for processing + XAI

public static void main(String[] args) {
    System.out.println("Starting Disease Prediction with Fog + Anchor XAI...");
    System.out.println("Initialising...");

    try {
        CloudSim.init(1, Calendar.getInstance(), false);
        String appId = "disease_pred";

        FogBroker broker = new FogBroker("broker");
        Application application = createApplication(appId, broker.getId());
        application.setUserId(broker.getId());

        // Create fog devices (cloud, fog servers, and computers)
        FogDevice cloud = createFogDevice("cloud", 44800, 40000, 100, 10000, 0, 0.01, 16 * 103, 16 * 83.25);
        cloud.setParentId(-1);
        fogDevices.add(cloud);

        // Create multiple fog servers
        for (int i = 0; i < numFogServers; i++) {
            FogDevice fogServer = createFogDevice("fog-server" + i, 6400, 8000, 10000, 10000, 1, 0.0, 107.339, 83.4333);
            fogServer.setParentId(cloud.getId());
            fogServer.setUplinkLatency(20);
            fogDevices.add(fogServer);


        // Assign computers to each fog server
        for (int j = 0; j < numComputers / numFogServers; j++) {
            int deviceId = i * (numComputers / numFogServers) + j;
            FogDevice computer = createFogDevice("computer-" + deviceId, 1000, 1000, 1000, 1000, 2, 0, 87.53, 82.44);
            computer.setParentId(fogServer.getId());
            computer.setUplinkLatency(5);
            fogDevices.add(computer);
        }
    }

    ModuleMapping moduleMapping = ModuleMapping.createModuleMapping();

    for (FogDevice device : fogDevices) {
        if (device.getName().startsWith("computer-")) {
            moduleMapping.addModuleToDevice("manual_input", device.getName());
            moduleMapping.addModuleToDevice("result_display", device.getName());
        } else if (device.getName().startsWith("fog-server")) {
            moduleMapping.addModuleToDevice("mlp_predictor", device.getName());
            moduleMapping.addModuleToDevice("anchor_explainer", device.getName());
        }
    }
    moduleMapping.addModuleToDevice("cloud_storage", "cloud");

    Controller controller = new Controller("controller", fogDevices, new ArrayList<>(), new ArrayList<>());
    controller.submitApplication(application, new ModulePlacementMapping(fogDevices, application, moduleMapping));

    TimeKeeper.getInstance().setSimulationStartTime(Calendar.getInstance().getTimeInMillis());
    CloudSim.startSimulation();
    CloudSim.stopSimulation();
```

Figure 20. iFogSim Simulation for Level-1 diagnosis (A) Source code in iFogSim (B) Result showing the energy consumption at each level.

(b)

```
Starting Disease Prediction with Fog + Anchor XAI...
Initialising...
Initialising...
Creating cloud_storage on device cloud
Creating manual_input on device computer-4
Creating result_display on device computer-4
Creating manual_input on device computer-5
Creating result_display on device computer-5
Creating mlp_predictor on device fog-server0
Creating anchor_explainer on device fog-server0
Creating manual_input on device computer-2
Creating result_display on device computer-2
Creating mlp_predictor on device fog-server1
Creating anchor_explainer on device fog-server1
Creating manual_input on device computer-3
Creating result_display on device computer-3
Creating manual_input on device computer-0
Creating result_display on device computer-0
Creating mlp_predictor on device fog-server2
Creating anchor_explainer on device fog-server2
Creating manual_input on device computer-1
Creating result_display on device computer-1
Starting CloudSim version 3.0
cloud is starting...
fog-server0 is starting...
computer-0 is starting...
computer-1 is starting...
fog-server1 is starting...
computer-2 is starting...
computer-3 is starting...
fog-server2 is starting...
computer-4 is starting...
computer-5 is starting...
0.0 Submitted application disease_pred
Entities started.
Simulation completed.
========================================
============== RESULTS ==================
========================================
EXECUTION TIME : 83
========================================
APPLICATION LOOP DELAYS
========================================
========================================
TUPLE CPU EXECUTION DELAY
========================================
========================================
cloud : Energy Consumed = 2692214.285714285
fog-server0 : Energy Consumed = 196748.725
computer-0 : Energy Consumed = 175060.0
computer-1 : Energy Consumed = 175060.0
fog-server1 : Energy Consumed = 196748.725
computer-2 : Energy Consumed = 175060.0
computer-3 : Energy Consumed = 175060.0
fog-server2 : Energy Consumed = 196748.725
computer-4 : Energy Consumed = 175060.0
computer-5 : Energy Consumed = 175060.0
Cost of execution in cloud = 40000.0
```

Figure 20. Continued.

is given tasks including manual input, result display, machine learning prediction (MLP predictor), and explanation creation (Anchor explainer). CloudSim 3.0 is used to execute the simulation after all entities have started, and this is where the disease prediction application is submitted and processed. The findings show that the computation within the distributed arrangement was relatively quick, with an execution time of 18 units. Details on energy usage are given for each device. The cloud uses the most energy (2,692,214.28 units), whereas computers and fog servers use much less (about 196,748 and 175,060 units, respectively), indicating the advantages of fog computing in terms of energy efficiency. Lastly, 40,000 units is the cost of execution in the cloud, which illustrates the financial trade-off between using distributed fog nodes and centralized cloud resources. This demonstrates how fog computing, while maintaining efficient prediction and explainability, can lower energy use and reliance on the cloud.

5. Conclusion

A complete fog computing framework has been suggested in this work for the dual level diagnosis of malaria. Malaria is completely manageable when there is timely treatment. The major problem found in the current scenario is delay in communication especially when the patient is in remote places. Fog computing framework reduces this delay in the communication and aids in timely treatment which eventually will save more lives. Reliable patient status reporting is provided by integrated architectures, and precise disease detection may be accomplished with the use of artificial intelligence algorithms. We have integrated AI with human-in-the-loop verification, which places medical professionals at the edge because certain cultures depend more on humans than on AI. Additionally, the majority of previous research focused on the initial stage of diagnosis, which aids in identifying just the most probable cases whereas serological test results are required to confirm the disease. In the fog computing environment, a lightweight model is also required. Our suggested framework with two lightweight models has addressed these issues. Optimized Lightweight MLP has been used in the level-1 diagnosis of the proposed framework. The healthcare expert who will serve as the middleman acquires the information after probable cases are quickly and efficiently identified. The Level-1 diagnosis model has been combined with Explainable AI (XAI) to help with the diagnosis. When compared to other ML/DL models, the suggested model outperformed them in terms of efficiency, achieving an impressive 96% accuracy rate. The ROC curve has been used to demonstrate the stability of the model. Strong model performance is indicated by the good AUC (Area Under the Curve) of 0.99 displayed by both the training and test curves. The healthcare

professional forwards the images of the serological tests of the probable cases to the second level of the framework for further checking. A lightweight customized CNN model aids in identifying the confirmed cases. When compared to other transfer learning models, transformer models, and DNN (Deep Neural Network) models, our proposed lite-weight CNN model showed greater efficiency with fewer parameters, a smaller model size and equivalent or more accuracy of 95% achieving a good balance between computational efficiency and performance. Finally, simulation using iFogSim has been carried out to check energy requirement for the level-1 diagnosis of Malaria.

A variety of datasets should be explored to train and evaluate the suggested models in the future, as the MLP model proposed for level-1 has been trained with only a small dataset. Similar simulation for level-2 can be carried out in future to measure the energy requirement. Training a model with large datasets improves model generalization, as this could significantly boost the effectiveness and promote model generalization. Also, synthetic data generation can be attempted in future to enhance the model. Efficient models that support 6G can be modelled in future. By implementing effective feature engineering techniques, the model's performance can be enhanced further. Future tests of the suggested fog architecture ought to be conducted in real-time situations. In the future, steps minimizing the cold start scenario as mentioned in the work (Golec et al., 2024) can be incorporated to overcome problems faced during real-time implementation. Also, a suitable simulation testbed for scalable, serverless AIoT infrastructures is provided in paper (Golec et al., 2024). For enhanced future development, cutting-edge methods like multimodal data fusion, block chain technology, smart contracts, federated learning, and transfer learning can be used.

Acknowledgements

Deepika R: Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Data curation, Conceptualization. **Pradeep Kumar T S:** review and editing, Supervision, Project administration, Conceptualization.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

The authors declared that the work is funded by Vellore Institute of Technology, Chennai.

Data availability statement

Datasets associated with this study has been found at https://pmc.ncbi.nlm.nih.gov/articles/PMC7093799/#sec1 and https://www.kaggle.com/datasets/iarunava/cell-images-for-detecting-malaria. Both are publicly available datasets and has appropriate access permission.

References

Adeboye, N. O., Abimbola, O. V., & Folorunso, S. O. (2020). Malaria patients in Nigeria: Data exploration approach. *Data in Brief*, *28*, 104997. https://doi.org/10.1016/j.dib.2019.104997

Ahmad, M., Qadri, S. F., Qadri, S., Saeed, I. A., Zareen, S. S., Iqbal, Z., & Mizanur Rahman, S. M. (2022). A lightweight convolutional neural network model for liver segmentation in medical diagnosis. *Computational Intelligence and Neuroscience*, *2022*(1), 7954333.

Ahmed, R., & Imran, A. S. (2024). Knee osteoarthritis analysis using deep learning and XAI on X-rays. *IEEE Access*, *12*, 68870–68879.

Al Muazzaz, H., & Goni, O. F. (2024, September). Detection of malaria parasite using lightweight CNN architecture and smart android application. *In 2024 IEEE International Conference on Power, Electrical, Electronics and Industrial Applications (PEEIACON)* (pp. 1-6). IEEE.

Anuradha, M., Jayasankar, T., Prakash, N. B., Sikkandar, M. Y., Hemalakshmi, G. R., Bharatiraja, C., & Britto, A. S. F. (2021). IoT enabled cancer prediction system to enhance the authentication and security using cloud computing. *Microprocessors and Microsystems*, *80*, 103301. https://doi.org/10.1016/j.micpro.2020.103301

Assaduzzaman, M., Bishshash, P., Nirob, M. A. S., Al Marouf, A., Rokne, J. G., & Alhajj, R. (2025). XSE-TomatoNet: An explainable AI based tomato leaf disease classification method using EfficientNetB0 with squeeze-and-excitation blocks and multi-scale feature fusion. *MethodsX*, *14*, 103159. https://doi.org/10.1016/j.mex.2025.103159

Ayalew, A. M., Admass, W. S., Abuhayi, B. M., Negashe, G. S., & Bezabh, Y. A. (2024). Smart malaria classification: A novel machine learning algorithms for early malaria monitoring and detecting using IoT-based healthcare environment. *Sensing and Imaging*, *25*(1), 55. https://doi.org/10.1007/s11220-024-00503-3

Azad, M., Khan, M. F. K., & Abd El-Ghany, S. (2025). XAI-enhanced machine learning for obesity risk classification: A stacking approach with LIME explanations. *IEEE Access*, *13*, 13847–13865.

Bader, M., Abdelwanis, M., Maalouf, M., & Jelinek, H. F. (2024). Detecting depression severity using weighted random forest and oxidative stress biomarkers. *Scientific Reports*, *14*(1), 16328. https://doi.org/10.1038/s41598-024-67251-y

Batool, A., & Byun, Y. C. (2025). A lightweight multi-path convolutional neural network architecture using optimal features selection for multiclass classification of brain tumor using magnetic resonance images. *Results in Engineering*, *25*, 104327. https://doi.org/10.1016/j.rineng.2025.104327

Benachour, Y., Flitti, F., & Khalid, H. M. (2025). Enhancing malaria detection through deep learning: A comparative study of convolutional neural networks. *IEEE Access*, *13*, 35452–35477.

Bhandari, M., Shahi, T. B., & Neupane, A. (2023). Evaluating retinal disease diagnosis with an interpretable lightweight CNN model resistant to adversarial attacks. *Journal of Imaging*, *9*(10), 219. https://doi.org/10.3390/jimaging9100219

Bhandari, M., Yogarajah, P., Kavitha, M. S., & Condell, J. (2023). Exploring the capabilities of a lightweight CNN model in accurately identifying renal abnormalities: Cysts, stones, and tumors, using LIME and SHAP. *Applied Sciences*, *13*(5), 3125. https://doi.org/10.3390/app13053125

Bria, Y. P., Yeh, C. H., & Bedingfield, S. (2021). Significant symptoms and nonsymptom-related factors for malaria diagnosis in endemic regions of Indonesia. *International Journal of Infectious Diseases*, *103*, 194–200. https://doi.org/10.1016/j.ijid.2020.11.177

Chattopadhay, A., Sarkar, A., Howlader, P., & Balasubramanian, V. N. (2018, March). Grad-cam++: Generalized gradient-based visual explanations for deep convolutional networks. In *2018 IEEE winter conference on applications of computer vision (WACV)* (pp. 839–847). IEEE.

Chaudhry, H. A. H., Farid, M. S., Fiandrotti, A., & Grangetto, M. (2024). A lightweight deep learning architecture for malaria parasite-type classification and life cycle stage detection. *Neural Computing and Applications*, *36*(31), 19795–19805. https://doi.org/10.1007/s00521-024-10219-w

Chen, S., Zhao, S., & Huang, C. (2023). An automatic malaria disease diagnosis framework integrating blockchain-enabled cloud–edge computing and deep learning. *IEEE Internet of Things Journal*, *10*(24), 21544–21553. https://doi.org/10.1109/JIOT.2023.3304526

Deepika, R., & Pradeep Kumar, T. S. (2025). Dual level dengue diagnosis using lightweight multilayer perceptron with XAI in fog computing environment and rule-based inference. *Scientific Reports*, *15*(1), 1–27. https://doi.org/10.1038/s41598-024-84936-6

Dheeraj, A., & Chand, S. (2024). LWDN: Lightweight DenseNet model for plant disease diagnosis. *Journal of Plant Diseases and Protection*, *131*(3), 1043–1059. https://doi.org/10.1007/s41348-024-00915-z

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., & Gelly, S. (2020). An image is worth 16×16 words: Transformers for image recognition at scale. arXiv:2010.11929.

Dwivedi, S., Pandey, K., Shubham, K., Pandey, O. J., Tripathi, A. M., Sandhan, T., & Hegde, R. M. (2025). MalaNet: A small world inspired neural network for automated malaria diagnosis. *IEEE Transactions on Artificial Intelligence*, 1–11.

Fang, X., Zhen, T., & Li, Z. (2023). Lightweight multiscale CNN model for wheat disease detection. *Applied Sciences*, *13*(9), 5801. https://doi.org/10.3390/app13095801

Fuhad, K. F., Tuba, J. F., Sarker, M. R. A., Momen, S., Mohammed, N., & Rahman, T. (2020). Deep learning based automatic malaria parasite detection from blood smear and its smartphone based application. *Diagnostics*, *10*(5), 329. https://doi.org/10.3390/diagnostics10050329

Gipiškis, R., Tsai, C. W., & Kurasova, O. (2024). Explainable AI (XAI) in image segmentation in medicine, industry, and beyond: A survey. *ICT Express*, *10*(6), 1331–1354.

Golec, M., Wu, H., Ozturac, R., Parlikad, A. K., Cuadrado, F., Gill, S. S., & Uhlig, S. (2024). Captain: a testbed for co-simulation of scalable serverless computing environments for AIOT enabled predictive maintenance in industry 4.0. *IEEE Internet of Things Journal*, 32283–32295.

Goni, M. O. F., & Mondal, M. N. I. (2023, June). Explainable AI based malaria detection using lightweight CNN. In *2023 International Conference on Next-Generation Computing, IoT and Machine Learning (NCIM)* (pp. 1–5). IEEE.

Gourisaria, M. K., Das, S., Sharma, R., Rautaray, S. S., & Pandey, M. (2020). A deep learning model for malaria disease detection and analysis using deep convolutional neural networks. *International Journal of Emerging Technologies*, *11*(2), 699–704.

Gupta, H., Vahid Dastjerdi, A., Ghosh, S. K., & Buyya, R. (2017). iFogSim: A toolkit for modeling and simulation of resource management techniques in the Internet of Things, Edge and Fog computing environments. *Software: Practice and Experience*, *47*(9), 1275–1296. https://doi.org/10.1002/spe.2509

Harmonis, J. A., Kusuma, S. A. F., Rukayadi, Y., & Hasanah, A. N. (2025). Exploring biomarkers for malaria: Advances in early detection and asymptomatic diagnosis. *Biosensors*, *15*(2), 106. https://doi.org/10.3390/bios15020106

Hasan, M. A., Haque, F., Sabuj, S. R., Sarker, H., Goni, M. O. F., Rahman, F., & Rashid, M. M. (2024). An end-to-end lightweight multi-scale CNN for the classification of lung and colon cancer with XAI integration. *Technologies*, *12*(4), 56. https://doi.org/10.3390/technologies12040056

https://www.kaggle.com/datasets/iarunava/cell-images-for-detecting-malaria

Iqbal, T., Khalid, A., & Ullah, I. (2024). Explaining decisions of a light-weight deep neural network for real-time coronary artery disease classification in magnetic resonance imaging. *Journal of Real-Time Image Processing*, *21*(2), 31. https://doi.org/10.1007/s11554-023-01411-7

Juyal, S., Sharma, S., & Shukla, A. S. (2021). Smart skin health monitoring using AI-enabled cloud-based IoT. *Materials Today: Proceedings*, *46*, 10539–10545. https://doi.org/10.1016/j.matpr.2021.01.074

Kaur, T., & Gandhi, T. K. (2019, December). Automated brain image classification based on VGG-16 and transfer learning. In *2019 international conference on information technology (ICIT)* (pp. 94–98). IEEE.

Kaushik, P., & Sharma, P. (2025, January). Deep learning-based malaria detection: A Robust CNN framework for accurate and automated diagnosis. In *2025 International Conference on Multi-Agent Systems for Collaborative Intelligence (ICMSCI)* (pp. 894–898). IEEE.

Kaushik, S., Lamba, A. K., Kansal, I., Khullar, V., & Sharma, P. (2024, December). Explainable deep learning for lung cancer detection: Comparing CNN and DenseNet201 with Grad-CAM. In *2024 2nd International Conference on Signal Processing, Communication, Power and Embedded System (SCOPES)* (pp. 1–5). IEEE.

Khan, S., Siddiqui, F., & Ahad, M. A. (2025). Bridging efficiency and interpretability: Explainable AI for multi-classification of pulmonary diseases utilizing modified lightweight CNNs. *Image and Vision Computing*, *185*, 105553. https://doi.org/10.1016/j.imavis.2025.105553

Khan, Z. A., Waqar, M., Raja, M. J. A. A., Chaudhary, N. I., Khan, A. T. M. A., & Raja, M. A. Z. (2025). Generalized fractional optimization-based explainable lightweight CNN model for malaria disease classification. *Computers in Biology and Medicine*, *185*, 109593. https://doi.org/10.1016/j.compbiomed.2024.109593

Koul, A., Bawa, R. K., & Kumar, Y. (2024). Enhancing the detection of airway disease by applying deep learning and explainable artificial intelligence. *Multimedia Tools and Applications*, *83*(31), 76773–76805. https://doi.org/10.1007/s11042-024-18381-y

Kumari, A., Tanwar, S., Tyagi, S., & Kumar, N. (2018). Fog computing for Healthcare 4.0 environment: Opportunities and challenges. *Computers & Electrical Engineering*, *72*, 1–13. https://doi.org/10.1016/j.compeleceng.2018.08.015

Kumari, S., Bhatia, M., & Stea, G. (2022). Fog-computing based healthcare framework for predicting encephalitis outbreak. *Big Data Research*, *29*, 100330. https://doi.org/10.1016/j.bdr.2022.100330

Lalithadevi, B., & Krishnaveni, S. (2024). Diabetic retinopathy detection and severity classification using optimized deep learning with explainable AI technique. *Multimedia Tools and Applications*, *83*(16), 1–65.

Lerma, M., & Lucas, M. (2022). Grad-CAM++ is equivalent to Grad-CAM with positive gradients. arXiv preprint arXiv:2205.10838.

Magotra, V., & Rohil, M. K. (2022). Malaria diagnosis using a lightweight deep convolutional neural network. *International Journal of Telemedicine and Applications*, *2022*(1), 4176982. https://doi.org/10.1155/2022/4176982

Mariki, M., Mkoba, E., & Mduma, N. (2022). Combining clinical symptoms and patient features for malaria diagnosis: machine learning approach. *Applied Artificial Intelligence*, *36*(1), 2031826. https://doi.org/10.1080/08839514.2022.2031826

Minh, Q. T., Phung, P. H., & Huu, P. N. (2023). Fog-enabled IoT framework for heart disease diagnosis systems. *Journal of Mobile Multimedia*, *19*(2), 389–418.

Mzoughi, H., Njeh, I., BenSlima, M., Farhat, N., & Mhiri, C. (2024). Vision transformers (ViT) and deep convolutional neural network (D-CNN)-based models for MRI brain primary tumors images multi-classification supported by explainable artificial intelligence (XAI). *The Visual Computer*, *41*(4), 1–20.

Najjar, F. H., Khudhair, K. T., Khudhair, Z. N., Alwan, H. H., & Al-khaykan, A. (2023, February). Acute lymphoblastic leukemia image segmentation based on modified HSV model. *Journal of Physics: Conference Series*, *2432*(1), 012020. https://doi.org/10.1088/1742-6596/2432/1/012020

Nema, S., Verma, A. K., Tiwari, A., & Bharti, P. K. (2021). Digital health care services to control and eliminate malaria in India. *Trends in Parasitology*, *37*(2), 96–99. https://doi.org/10.1016/j.pt.2020.11.002

Nettur, S. B., Karpurapu, S., Nettur, U., Gajja, L. S., Myneni, S., Dusi, A., & Posham, L. (2025). UltraLightSqueezeNet: A deep learning architecture for malaria classification with up to 54x fewer trainable parameters for resource constrained devices. arXiv preprint arXiv:2501.14172.

Pandey, A., Shivaji, B. A., Acharya, M., & Mohbey, K. K. (2024). Mitigating class imbalance in heart disease detection with machine learning. *Multimedia Tools and Applications*, *83*(34), 1–26.

Raisa, J. F., Rahman, M. S., Mahmud, I., Kaiser, M. S., & Han, D. S. (2025). Transition towards Healthcare 5.0: Impact of COVID-19 in the healthcare industry. *ICT Express*, *11*(3).

Salam, A., Hasan, S. N., Karim, M. J., Anower, S., Nahiduzzaman, M., Chowdhury, M. E., & Murugappan, M. (2024). Embedded system-based malaria detection from blood smear images using lightweight deep learning model. *International Journal of Imaging Systems and Technology*, *34*(6), e23205. https://doi.org/10.1002/ima.23205

Sawangarreerak, S., Thanathamathee, P., Lakkanawanit, P., & Wahab, N. S. A. (2025). Anchor-based explainable and causal artificial intelligence for enhancing financial predictions of future earnings. *IEEE Access*, *13*, 61026–61047.

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision* (pp. 618-626).

Shiuly, A., Dutta, D., & Mondal, A. (2022). Assessing compressive strengths of mortar and concrete from digital images by machine learning techniques. *Frontiers of Structural and Civil Engineering*, *16*(3), 347–358. https://doi.org/10.1007/s11709-022-0819-z

Siłka, W., Wieczorek, M., Siłka, J., & Woźniak, M. (2023). Malaria detection using advanced deep learning architecture. *Sensors*, *23*(3), 1501. https://doi.org/10.3390/s23031501

Singh, J., Sandhu, J. K., & Kumar, Y. (2024). Metaheuristic-based hyperparameter optimization for multi-disease detection and diagnosis in machine learning. *Service Oriented Computing and Applications*, *18*(1), 1–20.

Sora-Cardenas, J., Fong-Amaris, W. M., Salazar-Centeno, C. A., Castañeda, A., Martínez-Bernal, O. D., Suárez, D. R., & Martínez, C. (2025). Image-based detection and classification of malaria parasites and leukocytes with quality assessment of Romanowsky-stained blood smears. *Sensors*, *25*(2), 390. https://doi.org/10.3390/s25020390

Sukumarran, D., Loh, E. S., Khairuddin, A. S. M., Ngui, R., Sulaiman, W. Y. W., Vythilingam, I., & Hasikin, K. (2024). Automated identification of malaria-infected cells and classification of human malaria parasites using a two-stage deep learning technique. *IEEE Access*, *12*, 3459411.

Tan, M., & Le, Q. (2019, May). Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning* (pp. 6105–6114). PMLR.

Verma, P., Tiwari, R., Hong, W. C., Upadhyay, S., & Yeh, Y. H. (2022). FETCH: A deep learning-based fog computing and IoT integrated environment for healthcare monitoring and diagnosis. *IEEE Access*, *10*, 12548–12563. https://doi.org/10.1109/ACCESS.2022.3143793

Vijayakumar, V., Malathi, D., Subramaniyaswamy, V., Saravanan, P., & Logesh, R. (2019). Fog computing-based intelligent healthcare system for the detection and prevention of mosquito-borne diseases. *Computers in Human Behavior*, *100*, 275–285. https://doi.org/10.1016/j.chb.2018.12.009

Xing, X., Liang, G., Zhang, Y., Khanal, S., Lin, A. L., & Jacobs, N. (2022, March). Advit: Vision transformer on multi-modality pet images for Alzheimer disease diagnosis. In *2022 IEEE 19th International Symposium on Biomedical Imaging (ISBI)* (pp. 1–4). IEEE.

Yang, F., Poostchi, M., Yu, H., Zhou, Z., Silamut, K., Yu, J., & Antani, S. (2019). Deep learning for smartphone-based malaria parasite detection in thick blood smears. *IEEE Journal of Biomedical and Health Informatics*, *24*(5), 1427–1438. https://doi.org/10.1109/JBHI.2019.2939121

Yu, H. Q., & Reiff-Marganiec, S. (2021). Targeted ensemble machine classification approach for supporting IoT enabled skin disease detection. *IEEE Access*, *9*, 50244–50252. https://doi.org/10.1109/ACCESS.2021.3069024

Zhang, L., & Wu, X. (2025). The lightweight deep learning model in sunflower disease identification: A comparative study. *Applied Sciences*, *15*(4), 2104. https://doi.org/10.3390/app15042104

Zhang, Q., Bai, C., Yang, L. T., Chen, Z., Li, P., & Yu, H. (2019). A unified smart Chinese medicine framework for healthcare and medical services. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, *18*(3), 882–890. https://doi.org/10.1109/TCBB.2019.2914447